

Graph-Based Deep Learning for Malware Detection Using Windows Memory Forensics

Name: VIRAJ RAJESHBHAI CHUDASAMA

Supervisor: Mohamed Ghanem

Programme: MSc Cyber Security

Date: May 2026

Declaration

I declare that this project is my own work and has been prepared in line with the rules and guidance of the University of Wales Trinity Saint David.

Signed: _____

Date: _____

Abstract

Malware often hides in live memory, while file-based scanning can miss injection, covert processes, and short-lived network activity. Windows memory forensics captures processes, DLLs, memory regions, handles, and network objects at capture time, but Volatility plugin output is large and difficult to compare across samples by hand.

This dissertation implements a proof-of-concept pipeline: Volatility 3 artefacts are converted into typed memory-snapshot graphs, scored by graph neural networks (GIN, GraphSAGE, GAT, GINE), and explained with structured evidence and uncertainty fields. A hybrid rule layer supplies graph attributes and triage verdicts; binary and two-model paths support analyst review.

The central empirical finding is label–signal mismatch on benign baselines: in the current manifest ($n=30$), **14 of 15** benign-labelled (NoVirus) rows are flagged `uncertain=True` because rule-based triage still reports HIGH or CRITICAL activity (injection counts, process scores, C2-style connections). Folder names are therefore an imperfect proxy for ground truth; uncertainty must be reported, not hidden.

Contributions include a reproducible extraction-to-graph workflow, manifest governance, GNN training scripts with family-grouped cross-validation, and optional dashboard scanning. Metrics tables in Chapter 6 use placeholders until grouped CV is run on archived checkpoints.

Limitation: With $n=30$ graphs and noisy benign labels, the work demonstrates feasibility and evidence organisation—not production detection performance.

Keywords: malware detection, Windows memory forensics, Volatility 3, graph neural networks, heterogeneous graphs, PyTorch Geometric, GINE, anomaly detection, digital forensics, explainable AI

Contents

Declaration	1
Abstract	2
1 Introduction	8
1.1 Background	8
1.2 Motivation	9
1.3 Problem Statement	9
1.4 Research Aim	10
1.5 Key Definitions	10
1.6 Primary Expected Contribution	10
1.7 Research Objectives	10
1.8 Research Questions	11
1.9 Contributions Preview	11
1.10 Scope	12
1.11 Dissertation Structure	12
1.12 Context: Memory Forensics in Incident Response	12
1.13 Expected Outcomes	13
2 Literature Review	14
2.1 Malware Detection	14
2.2 Static, Dynamic, and Memory-Based Analysis	15
2.3 Memory Forensics	15
2.4 Windows Memory Artefacts and Internals	16
2.5 Standards, Frameworks, and Incident Response Context	17
2.6 Graph Representation Learning	17
2.6.1 Message passing intuition	18
2.6.2 Limitations of GNNs in security applications	18
2.6.3 Dataset leakage and evaluation validity	19
2.6.4 Adversarial machine learning in malware detection	19
2.7 Ransomware and Memory-Resident Behaviour	19

2.8	Graph-Based Malware Detection	20
2.9	Anomaly Detection and One-Class Learning	20
2.10	Evaluation Metrics in Security Machine Learning	21
2.11	Explainability and Calibration	21
2.12	Research Gap	21
2.13	Machine Learning Tooling for Security Research	22
2.14	Synthesis for This Dissertation	22
3	Research Methodology	23
3.1	Research Design	23
3.2	Data Sources	23
3.3	Volatility 3 Extraction	24
3.4	Graph Construction Method	25
3.5	Feature Engineering	26
3.6	Model Methodology	26
3.7	Evaluation Method	27
3.8	Ethics	28
3.9	Data Collection and Sample Selection	28
3.10	Labelling Policy and Uncertainty Handling	28
3.11	Detailed Feature Groups	29
3.12	Training and Validation Protocol	29
3.13	Threats to Validity	29
3.14	Research Timeline and Artefact Milestones	30
3.15	Relation to <code>docs.md</code> and Code Contracts	30
4	System Design	31
4.1	Overview	31
4.2	Extraction Layer	31
4.3	Graph Layer	31
4.4	Triage and Graph Attributes	33
4.5	Dataset Manifest Design	33
4.6	Model Stack	34
4.7	Evidence Design	34
4.8	Interface Between Triage Rules and Learning	35
4.9	Dashboard and Operational Integration	35
4.10	Design Rationale by Layer	35
4.10.1	Why separate extraction and learning	35
4.10.2	Why use both rules and GNNs	35
4.10.3	Why graph-level classification	36

4.11	Failure Handling and Health Flags	36
4.12	Security and Privacy Considerations in Design	36
5	Implementation	37
5.1	Development Environment	37
5.2	Main Repository Components	37
5.2.1	Extraction and graph build	38
5.2.2	Dataset and model code	38
5.3	Data Contracts	38
5.4	Node and Edge Implementation	39
5.5	Why GINE and What Alternatives Were Considered	39
5.5.1	Why GINE over untyped convolutions	39
5.5.2	Why graph-level classification, not node labels	39
5.5.3	Supported backbones and expected trade-offs	40
5.5.4	What did not work well in practice	40
5.6	GNN Implementation	40
5.6.1	Training configuration	41
5.7	Evidence and Inference Implementation	41
5.8	Implementation Challenges	42
5.9	Reproducibility	42
5.10	Per-Module Implementation Notes	42
5.10.1	<code>build_graph.py</code>	42
5.10.2	<code>filter_malicious.py</code>	42
5.10.3	<code>analyze_graph.py</code>	43
5.10.4	<code>dataset.py</code> and training scripts	43
5.10.5	<code>server.py</code> and <code>model_scan.py</code>	43
5.11	Hyperparameters and Training Defaults	43
5.12	Performance and Resource Considerations	44
5.13	Testing and Validation During Development	44
6	Evaluation and Discussion	45
6.1	Central Empirical Finding	45
6.2	Proof-of-Concept Scope	46
6.3	Descriptive Corpus Statistics	46
6.4	Rule-Based Triage Analysis	47
6.5	Quantitative Machine Learning Evaluation	49
6.5.1	GNN backbone comparison (StratifiedGroupKFold by family)	49
6.5.2	Tabular baselines (manifest graph-level features)	49
6.5.3	Cross-validation protocol	49

6.5.4	Evaluation figures (placeholders until predictions exist)	49
6.6	Failure Cases and Edge Families	50
6.6.1	W32.MyDoom.A. pair (very small graphs)	50
6.6.2	Cerber pair (similar scale, divergent C2 counts)	50
6.6.3	GoldenEye pair (high injection on both rows)	50
6.6.4	WannaCry case study	50
6.6.5	When a GNN may not help	51
6.7	Binary and Two-Model Paths	51
6.8	Explainability	52
6.9	What Results Prove and Do Not Prove	52
6.10	Limitations	52
6.11	Practical Value	53
6.12	Summary	53
7	Conclusion and Future Work	54
7.1	Conclusion	54
7.2	Answers to Research Questions	54
7.3	Reflection on the Research Aim	55
7.4	Contributions	55
7.5	Future Work	55
7.6	Implications for Practice	56
7.7	Final Statement	56
8	Reflection	57
8.1	Project Learning	57
8.2	Technical Reflection	57
8.3	Research Reflection	58
8.4	Professional Reflection	58
8.5	Closing Reflection	58
8.6	Skills Developed	58
8.7	What I Would Do Differently	59
8.8	Personal Outcome	59
A	Project Artefacts and Reproducibility	60
A.1	Project Artefact Map	60
A.2	Manifest Column Glossary	60
A.3	Core Plugin Set	61
A.4	Reproducibility Commands	61
A.5	Acronyms	62
A.6	Manifest Excerpt (First Five Rows)	62

A.7	Figure Generation	62
B	Dataset Statistics Summary	63
B.1	Label and uncertainty counts	63
B.2	Graph size ranges	63
B.3	Families represented	63
B.4	Interpretation notes	64
B.5	Recommended Reporting Checklist for Future Experiments	64
C	Glossary of Key Terms	65
D	Volatility Plugin Notes (Extended)	66

Chapter 1

Introduction

1.1 Background

Cybercrime and cyber-enabled fraud continue to affect global economies. Malware is a core tool in many attack chains, including ransomware campaigns, credential theft, and long-term compromise. Organisations therefore invest in endpoint detection, logging, and forensic response capabilities. Despite these investments, skilled analysts remain necessary because automated tools can miss context, produce false positives, or fail on novel families.

Malware is still a major problem for organisations, universities, public services, and personal users [1]. Modern malware does not only exist as a file on disk. It may inject code into trusted processes, load suspicious modules, communicate with remote hosts, hide process activity, or perform actions only for a short time [2]. These behaviours are often visible in memory before they are fully visible in normal file logs.

Attackers also use techniques described in public frameworks such as MITRE ATT&CK, including process injection and impact operations [3], [4]. Security teams therefore need methods that can inspect live system state, not only files after the event.

Digital forensics and incident response teams use memory forensics to investigate this type of activity. Memory forensics can show active processes, terminated process traces, DLLs, network connections, command-line arguments, file handles, registry hives, and suspicious memory pages [5]. Tools such as Volatility 3 make this analysis possible in a structured way [6]. Even so, the output of many plugins can be difficult to read. A single memory dump can produce many CSV files and thousands of rows.

This project uses graph-based deep learning to improve how memory-forensics artefacts are represented and analysed. A graph is a natural format for memory evidence because operating system behaviour is relational [7]. A process starts another process. A process loads a DLL. A memory region belongs to a process. A network connection is owned by a process. These relationships are important for malware detection. A flat table can store these values, but it does not show the structure as clearly as a graph.

Graph neural networks can learn from both node features and graph structure [8]. This makes them suitable for malware detection from memory snapshots. Instead of checking only one indicator, the model can learn patterns across process trees, memory regions, network objects, and other forensic artefacts.

1.2 Motivation

The motivation for this work comes from three practical problems seen during project development.

First, memory-forensics output is large. Analysts can spend significant time moving between plugin tables to build a mental model of one host. A graph representation can keep relationships visible in one structure.

Second, malware labels are not always clean. Benign memory snapshots may still contain suspicious indicators such as injected pages or high process scores. A detection system should report uncertainty instead of hiding it [9].

Third, research prototypes often lack reproducibility. Without manifests, contracts, and saved artefacts, it is hard to repeat experiments or explain a score months later [10]. This dissertation therefore treats the pipeline and dataset manifest as core research outputs, not only the model weights.

1.3 Problem Statement

The main problem is that Windows memory forensics produces rich evidence, but this evidence is hard to scale, compare, and interpret manually. Analysts can run Volatility plugins, but they still need to connect many separate outputs into one clear view of behaviour. This becomes harder when there are many samples or when the evidence is noisy.

This dissertation addresses four practical gaps:

- **Representation gap:** plugin outputs are usually stored as separate tables, while malware behaviour is relational.
- **Learning gap:** many forensic workflows use rules or manual checks, but they do not learn structural patterns from complete memory snapshots.
- **Explainability gap:** a useful detection system should give evidence, not only a malware or benign label.
- **Reproducibility gap:** memory-forensics experiments need stable artefacts, manifests, data contracts, and repeatable commands.

1.4 Research Aim

The aim of this dissertation is to design and evaluate a graph-based deep learning pipeline for malware detection using Windows memory-forensics artefacts extracted with Volatility 3.

The work is positioned as a **proof-of-concept**: it demonstrates feasibility of graph-based memory-forensics machine learning with structured evidence and uncertainty reporting, not production-grade detection performance.

1.5 Key Definitions

For clarity, this dissertation uses the following working definitions:

Memory snapshot graph: a heterogeneous graph built from one memory dump’s Volatility artefacts, representing system entities and relationships at capture time.

Malware-labelled sample: a manifest row with label 1, usually corresponding to a WithVirus folder.

Benign-labelled sample: a manifest row with label 0, usually corresponding to a NoVirus folder, possibly flagged uncertain.

Evidence: structured fields (attributes, nodes, edges, narratives) that explain why a sample received a score or triage state.

1.6 Primary Expected Contribution

The expected primary contribution is empirical and methodological rather than a new detection benchmark. On the available corpus, benign-labelled memory snapshots frequently exhibit HIGH or CRITICAL triage signals despite NoVirus folder names (14 of 15 benign rows marked uncertain in the manifest). This finding shows that memory-forensics labels derived from folder pairing can diverge sharply from rule-based risk signals.

The secondary contributions are engineering artefacts: a Volatility-to-graph pipeline, manifest-indexed dataset, GNN and tabular evaluation scripts with family-grouped splits, and JSON evidence fields for analyst review. Together they support reproducible feasibility studies on small memory-forensics corpora.

1.7 Research Objectives

The objectives are:

1. To extract Windows memory-forensics artefacts from memory dumps using Volatility
3. This includes process, memory, network, and registry-related plugin outputs

stored as repeatable CSV artefacts.

2. To convert plugin outputs into typed memory-snapshot graphs with entities and relationships analysts recognise (process parentage, DLL loading, injection links).
3. To engineer node, edge, and graph-level features suitable for graph neural networks. Features must remain stable across samples so training and inference use the same contracts.
4. To train and analyse GNN-based malware detection models. The project includes binary classification and one-class conformity models for richer triage states.
5. To preserve model evidence and uncertainty so that analyst review remains possible. Outputs should include top attributes, important nodes, and short narratives.
6. To evaluate the strengths and limits of the approach using available project artefacts. Evaluation must discuss dataset size, label noise, and family overlap honestly.

1.8 Research Questions

This project is guided by the following questions:

1. How can Windows memory-forensics artefacts be represented as a graph for malware detection? The answer is developed through the node and edge schema in Chapters 3 and 4, and through graph statistics in Chapter 6.
2. Which system entities and relationships are most useful for graph-based detection? The project prioritises process, memory, network, and DLL relationships, plus behaviour-intent edges aligned with common attack patterns.
3. How can GNN models support malware detection while still providing structured evidence? The binary and two-model paths combine learned scores with rule-based graph attributes and JSON evidence fields.
4. What limitations appear when graph-based learning is applied to a small memory-forensics corpus? Chapter 6 discusses uncertain benign rows, graph-size variation, and the risk of overconfident conclusions.

1.9 Contributions Preview

The main contributions of this dissertation are:

- a Volatility 3 extraction workflow for Windows memory artefacts;

- a heterogeneous graph schema for memory-forensics entities and relationships;
- graph-level feature engineering linked to triage signals;
- GNN model support for binary and one-class analysis paths;
- structured model evidence and uncertainty reporting;
- a dataset manifest for reproducible experiments;
- optional dashboard integration for upload-session model scanning.

1.10 Scope

This is a writing and research project based on an implemented repository. The work focuses on Windows memory dumps, Volatility 3 artefacts, graph construction, and graph-based deep learning. The dataset includes several malware-family folders and paired NoVirus folders. Ransomware samples are included as part of the corpus, but the topic is not limited to ransomware. The correct topic is broader: graph-based deep learning for malware detection using Windows memory forensics.

The dissertation does not include malware binaries in the written submission. It discusses memory dumps only at the artefact level. Ethics and safe handling are addressed in Chapter 3.

1.11 Dissertation Structure

Chapter 1 introduces the problem and the research aim. Chapter 2 reviews memory forensics, malware analysis, graph learning, GNNs, anomaly detection, and explainability. Chapter 3 explains the methodology. Chapter 4 presents the system design. Chapter 5 describes the implementation. Chapter 6 discusses the results and limitations. Chapter 7 concludes the dissertation and suggests future work. Chapter 8 gives a personal reflection.

1.12 Context: Memory Forensics in Incident Response

Incident response workflows often move from live collection to disk forensics and then to memory forensics when analysts suspect hidden processes, injection, or ransomware activity [11]. Commercial and open-source tools can acquire RAM images, but the analysis step remains difficult at scale. Analysts must decide which processes, memory regions, and network objects deserve deeper review.

This dissertation does not replace Volatility or other forensic suites. Instead, it proposes a structured bridge between plugin outputs and machine-learning models. The bridge is a

graph representation with explicit evidence fields. That design choice keeps the workflow understandable for security practitioners who may not be deep-learning specialists.

1.13 Expected Outcomes

By the end of this work, a reader should understand four outcomes:

1. how Windows memory artefacts are converted into heterogeneous graphs;
2. how graph features and GNN models are used for malware triage;
3. how uncertainty and evidence are reported in analysis outputs;
4. what limitations apply when the corpus is small and labels are noisy.

These outcomes align with the research questions and support a fair interpretation of results in later chapters. Together they define what success means for this applied security project.

Chapter 2

Literature Review

2.1 Malware Detection

Malware remains a persistent threat to individuals, businesses, and critical infrastructure. Attackers use packed binaries, living-off-the-land techniques, credential theft, ransomware, and memory-resident methods to evade traditional defences. Public guidance from CISA and related bodies emphasises layered detection, incident response preparation, and malware analysis tradecraft [12], [13].

Malware detection has moved from simple signature matching to more behaviour-based and learning-based approaches. Signature methods can be fast, but they often fail when malware changes its hash, packs itself, or uses obfuscation [14]. Behaviour-based approaches study actions such as process creation, API use, file activity, persistence, network communication, and memory injection. These methods are more useful when malware changes its surface form but keeps similar behaviour.

Christodorescu et al. introduced semantics-aware malware detection to reason about program meaning rather than raw bytes alone [15]. Moser et al. showed that static analysis alone has clear limits for packed or obfuscated malware [16]. These findings explain why modern work often combines multiple evidence layers instead of relying on one technique.

Dynamic malware analysis is useful because it observes behaviour during execution. Egele et al. describe how automated dynamic analysis systems can record system calls, network traffic, and other runtime events [2]. Bayer et al. show that malware behaviour can be clustered and compared using runtime profiles [17]. Willems et al. discuss automated dynamic malware analysis using sandbox environments [18]. Yin et al. present Panorama, which captures system-wide information flow for malware detection and analysis [19]. These ideas support the approach used in this dissertation because memory forensics also studies runtime state.

However, dynamic analysis can be limited by sandbox evasion, environment differences, and incomplete execution. Memory forensics gives a different evidence layer. It can

capture live runtime objects and artefacts that may not exist on disk. This is useful for malware that hides itself or runs only in memory.

Ucci et al. provide a broad survey of machine learning techniques for malware analysis [1]. Their review shows that feature design, label quality, and evaluation design are often more important than the choice of classifier. This dissertation follows that lesson by focusing on graph representation and reproducible artefacts before claiming strong detection performance.

2.2 Static, Dynamic, and Memory-Based Analysis

Table 2.1 compares three common analysis layers. Each layer has strengths, but none is sufficient alone for difficult malware cases.

Table 2.1: Comparison of malware analysis layers relevant to this project.

Layer	Typical data	Uses graph?	Explainability	Main limitation
Static	Bytecode, CFG, API calls	Often	Medium	Packing and obfuscation
Dynamic	Syscalls, files, network	Sometimes	Medium	Sandbox evasion
Memory	Processes, VAD, sockets	Rare in prior work	High potential	Noisy baselines, scale

Static analysis is efficient for known patterns, but attackers can change file structure while keeping malicious runtime behaviour. Dynamic analysis observes execution, yet it may miss behaviour that appears only in specific host states. Memory forensics captures volatile state at one point in time. Ligh et al. explain that memory analysis can reveal artefacts not available from disk alone [5]. This project uses memory forensics as the primary evidence source and then converts artefacts into graphs for learning.

2.3 Memory Forensics

Memory forensics is the analysis of volatile memory. It is important because memory can contain processes, handles, command lines, loaded modules, sockets, registry artefacts, injected pages, encryption keys, and other evidence. NIST recommends forensic techniques as part of incident response and evidence handling [20]. NIST also provides guidance for security incident handling in operational environments [11].

Volatility is a common framework for memory forensics. Volatility 3 is a modern rewrite of the framework and supports many plugins for Windows, Linux, and macOS memory analysis [6], [21]. In this project, Volatility 3 is used to extract Windows artefacts such

as `pslist`, `psscan`, `pstree`, `malfind`, `netscan`, `cmdline`, `dlllist`, `handles`, `threads`, `vadinfo`, `filescan`, `driverscan`, `ssdt`, and registry hive information.

Case and Richard discuss the future direction of memory forensics research and highlight the need for scalable, repeatable workflows [22]. Garfinkel also argues that digital forensics research must improve reproducibility and validation over the next decade [23]. These points motivate the manifest-based pipeline used in this dissertation.

Memory forensics is not simple. Symbol tables, plugin differences, large dump sizes, and noisy outputs can affect results. Therefore, this project includes a pipeline that stores outputs, checks artefacts, builds graphs, and records dataset-level metadata.

Analysts must also understand acquisition effects. A memory image captured too late may miss short-lived processes. An image captured during heavy system load may contain many benign artefacts that look suspicious in isolation. These practical constraints are one reason why graph-level uncertainty reporting is included in the project design.

Compared with disk forensics, memory forensics is closer to runtime behaviour. Compared with sandbox logs, memory forensics can show state that malware tried to hide after execution. The dissertation uses this middle layer as the primary evidence source because it matches the project’s Windows memory dump data.

2.4 Windows Memory Artefacts and Internals

Windows is a widely deployed operating system with complex kernel and user-mode internals. Forensic analysts commonly examine processes, threads, VAD trees, loaded modules, network connections, and kernel structures when investigating compromise. The Volatility plugin set used in this project reflects common investigative questions: what was running, what was connected, what was injected, and what kernel objects were present at capture time.

Windows memory contains many object types that are useful for malware detection. Process lists show visible processes. Pool scans can show hidden or terminated process traces. Command-line artefacts can show suspicious execution. DLL lists can show unusual module loading. Memory regions can show executable or writable pages. Network scans can show active connections. Handles can show process ownership and access behaviour.

Russinovich et al. describe Windows internals in detail, including process management, memory management, and I/O subsystems [7], [24]. Microsoft documentation also provides practical reference material for analysts [25]. This background helps explain why memory snapshots contain rich relational structure rather than isolated records.

The important point is that these artefacts are connected. A suspicious process may have injected memory, unusual DLL paths, many handles, and network activity. A graph can join these signals into one structure. This is why the dissertation uses graph construction instead of only table-based analysis.

Graziano et al. study hypervisor memory forensics and show that memory analysis can extend beyond standard host snapshots [26]. Although this project focuses on Windows user-mode and kernel artefacts from standard dumps, that work supports the broader claim that memory evidence remains central in advanced threat analysis.

2.5 Standards, Frameworks, and Incident Response Context

Digital investigations must follow sound evidence handling. ISO/IEC 27037 provides guidelines for identification, collection, acquisition, and preservation of digital evidence [27]. Casey discusses digital evidence and computer crime from a forensic science perspective [28]. Carrier provides detailed file system forensic analysis methods [29]. These sources do not define malware models directly, but they justify careful artefact management in this project.

The MITRE ATT&CK framework documents adversary tactics and techniques in a structured way [3]. Techniques such as process injection (T1055) and data encrypted for impact (T1486) are relevant to memory-forensics findings [4], [30]. The project maps some graph edge types to these ideas so that model inputs are easier to interpret during analyst review.

2.6 Graph Representation Learning

Graphs are used when relationships are as important as individual records. In a graph, nodes represent entities and edges represent relationships. Graph representation learning tries to learn useful vector representations from graph structure and node attributes. Hamilton gives a broad introduction to graph representation learning and its use in machine learning [8].

Before GNNs became common, methods such as DeepWalk and node2vec learned embeddings from random walks on graphs [31], [32]. Mikolov et al. introduced efficient word embedding methods that influenced later graph embedding work [33]. These methods show that structure can be converted into vectors, but they do not always use node attributes or edge types as flexibly as modern GNNs.

Graph neural networks extend deep learning to graph data. Kipf and Welling introduced a graph convolutional network for semi-supervised learning on graphs [34]. Hamilton, Ying, and Leskovec introduced GraphSAGE, which learns inductive node representations by aggregating neighbourhood information [35]. Velickovic et al. introduced graph attention networks, where attention weights are used for neighbouring nodes [36]. Xu et al. introduced the Graph Isomorphism Network and studied the expressive power of GNNs [37].

Fey and Lenssen describe practical graph learning with PyTorch Geometric [38]. These models support the idea that structure can improve learning.

This project uses GNN ideas for graph-level malware detection. The model does not classify one node only. It scores the whole memory snapshot graph.

2.6.1 Message passing intuition

In message-passing GNNs, each node updates its representation using messages from neighbours. After several layers, a node representation can reflect multi-hop structure. For memory graphs, this means a process node can be influenced by connected memory regions, DLLs, and network objects. Graph-level pooling then aggregates node information into a snapshot-level embedding.

The choice of backbone involves trade-offs. GIN is strong for discriminative graph classification. GraphSAGE supports inductive settings. GAT adds attention weights to neighbours. GINE is attractive for this project because edge types are informative and should not be collapsed into unlabeled edges. The implementation therefore emphasises GINE-style models for binary and one-class paths.

2.6.2 Limitations of GNNs in security applications

GNNs are not a default win for forensic malware detection. Several structural limits apply.

Oversmoothing and depth. Deep message-passing stacks can make node representations indistinguishable, reducing discriminative power on large graphs [39], [40]. Memory snapshots with tens of thousands of nodes are therefore sensitive to layer count and pooling choices.

Scalability. Full-graph training on large memory graphs is memory-intensive. Sub-sampling, pooling, or graph coarsening may be required, which can remove rare but important artefacts (for example a single injected region).

Explainability gaps. Post-hoc explainers (GNNExplainer, SHAP) help, but they do not guarantee faithful forensic narratives [41], [42]. This project mitigates the gap with rule-derived attributes and explicit evidence JSON, not with explanation methods alone.

Adversarial manipulation. Attackers can perturb graph structure or features—for example by hiding processes, splitting activity across benign parents, or injecting decoy edges [43], [44]. A model trained on laboratory families may not resist adaptive evasion.

GNNs do not remove the need for sound data collection. Incomplete plugin output yields incomplete graphs. Noisy labels induce spurious correlations. These limits motivate grouped evaluation and uncertainty reporting in Chapter 6.

2.6.3 Dataset leakage and evaluation validity

Sommer and Paxson argue that security machine-learning papers often overstate results because train and test distributions overlap or because labels do not reflect deployment base rates [45]. In malware research, family-level leakage is common: variants from the same family share packers, C2 patterns, or host profiles. Random row splits can inflate metrics when WithVirus and NoVirus pairs from one family appear in both train and test.

This dissertation uses `family` as the grouping key for `StratifiedGroupKFold`, matching the paired-folder structure of the corpus. Even with grouped splits, $n=30$ yields high variance; metrics should be reported with fold detail and interpreted as feasibility evidence, not operational guarantees [46].

2.6.4 Adversarial machine learning in malware detection

Ucci et al. survey ML for malware analysis and note that feature design and evaluation matter as much as model choice [1]. Apruzzese et al. emphasise that real attackers may avoid gradient-based attacks yet still evade detectors through distribution shift, label noise, and concept drift [46]. Pierazzi et al. show that many published defences fail under realistic threat models [47].

For memory-forensics graphs, adversarial risk includes both ML evasion and forensic anti-analysis (process hiding, timestomping, memory wiping). A graph model may appear accurate on paired lab folders while failing on enterprise baselines with security tools and administrative noise—the uncertain benign rows in this project’s manifest illustrate that overlap [48].

2.7 Ransomware and Memory-Resident Behaviour

Several families in the project corpus are ransomware-related (for example WannaCry, Cerber, GandCrab, and Locky variants). Ransomware often combines encryption behaviour with process injection and network activity. MITRE technique T1486 (data encrypted for impact) is relevant when interpreting ransom-note signals and high-impact process scores [30].

Memory forensics can capture pre-encryption activity that disk-only analysis misses. However, ransomware detection from memory is still difficult because benign administrative tools and security software can produce overlapping indicators. This supports the dissertation’s focus on triage and evidence rather than automatic blocking.

2.8 Graph-Based Malware Detection

Graph-based malware detection has been studied in several forms. Some work uses function call graphs, control-flow graphs, or API-call graphs from binaries. Other work uses system-call graphs or process interaction graphs. Rieck et al. learn and classify malware behaviour from dynamic analysis traces [49]. Arp et al. present DREBIN, an explainable Android malware detection approach based on structural features [50]. These approaches show that graph structure can capture behaviour that simple strings or hashes cannot.

This dissertation differs from static binary graph work because it builds graphs from memory-forensics artefacts. The graph represents a whole Windows system snapshot, not only one executable file. This matters because malware behaviour often appears through interactions between many system objects. A single malicious file may be hidden, but its process, memory, network, and handle relationships can still be visible.

2.9 Anomaly Detection and One-Class Learning

Anomaly detection treats deviant behaviour as rare events in a background distribution. In network security, Sommer and Paxson argued that ML evaluation must reflect operational base rates and adversarial adaptation [45]. In malware analysis, anomaly detection can support finding previously unseen families when training data does not cover all future threats.

Malware datasets are often imbalanced and incomplete. It can be hard to collect enough clean and malicious examples for every family. One-class learning is useful in this setting because a model can learn the shape of one class and measure how closely a new sample fits it. Ruff et al. introduced Deep SVDD for deep one-class classification [51]. Pang et al. provide a review of deep anomaly detection methods [52].

Sommer and Paxson warn that machine learning for intrusion detection must account for distribution shift and evaluation pitfalls [45]. Axelsson discusses the base-rate fallacy and its effect on detection difficulty [53]. These papers are important for this dissertation because memory-forensics datasets often contain rare events and noisy benign baselines.

This project includes a two-model analysis design. One model learns malware conformity and another model learns benign conformity. The outputs are combined with heuristic risk signals. This design is useful because it can show uncertain cases instead of forcing every sample into a simple binary decision.

2.10 Evaluation Metrics in Security Machine Learning

Security papers often report accuracy, precision, recall, F1, ROC-AUC, and false-positive rates. Davis and Goadrich explain relationships between precision-recall and ROC views [54]. In imbalanced settings, accuracy alone can be misleading because most samples may be benign. The base-rate fallacy literature shows why rare-event detection is difficult even with reasonable classifiers [53].

For this dissertation, precision and recall are still relevant, but they must be interpreted with label noise and family leakage in mind. A high recall on a small validation fold may not generalise to new families. A low false-positive rate on clean enterprise hosts is often more important in operations than a small F1 gain on a lab corpus.

Calibration is a separate issue. Even when discrimination is good, probability outputs may be overconfident [9]. That is why the project discusses calibrated probabilities and ambiguous states in the binary analysis path.

2.11 Explainability and Calibration

Cybersecurity models should be explainable because analysts need to justify decisions. A model that returns only a probability may not be enough for incident response. Doshi-Velez and Kim argue that interpretability should be treated as an important research problem [55]. Ying et al. introduced GNNExplainer to explain predictions from graph neural networks [41]. Ribeiro et al. propose LIME for explaining classifier predictions [56]. Lundberg and Lee introduce SHAP as a unified approach to model interpretation [42]. Shrikumar et al. describe DeepLIFT for feature importance propagation [57]. Although this project does not fully implement all of these tools, it preserves evidence fields such as important graph attributes, top nodes, and important edge pairs.

Calibration is also important. Guo et al. showed that modern neural networks can be miscalibrated [9]. Davis and Goadrich explain the relationship between precision-recall and ROC curves [54]. A model may be accurate but still overconfident. This is risky in malware detection. The project therefore includes calibrated probability fields and ambiguity states in the binary analysis path.

2.12 Research Gap

The literature shows strong work in memory forensics, malware analysis, graph learning, and anomaly detection. However, there is still a gap between memory-forensics artefact extraction and graph-based deep learning with analyst-readable evidence. Many studies focus on static binaries or isolated runtime logs. Few studies provide a full pipeline

that includes extraction, graph construction, dataset manifests, graph-level learning, uncertainty reporting, and optional operational scanning in one reproducible project.

This dissertation contributes by joining Volatility 3 extraction, heterogeneous graph construction, graph-level GNN learning, uncertainty handling, and operational reporting in one applied project. The next chapters describe how the pipeline was designed, implemented, and evaluated on the available memory-forensics corpus.

2.13 Machine Learning Tooling for Security Research

Several general-purpose libraries support the implementation side of this work. PyTorch provides the core deep learning framework [58]. PyTorch Geometric extends it for graph data [38], [59]. Scikit-learn remains a common baseline toolkit for tabular malware features [60]. Pandas, NumPy, and NetworkX support data processing and graph construction [61], [62], [63].

FastAPI is used in the project server layer for HTTP-based orchestration [64]. While framework choice is not the research contribution, these tools reduce engineering friction and make the pipeline easier to reproduce.

2.14 Synthesis for This Dissertation

The literature review suggests four design principles for this project.

First, memory evidence is relational and should be stored as a graph rather than only as isolated plugin tables. **Second**, learning methods must handle small, noisy datasets and report uncertainty openly. **Third**, explainability is not optional in security operations; evidence fields should accompany scores. **Fourth**, reproducibility requires manifests, contracts, and versioned artefacts.

Chapter 3 translates these principles into a concrete methodology. Chapter 4 and Chapter 5 describe the system realisation. Chapter 6 tests the claims against project data and discusses limitations without overstating detection performance.

Finally, the literature review highlights a simple practical lesson: in memory-forensics ML, data engineering and evaluation design are not supporting tasks. They are core research contributions. The novelty of this dissertation is therefore not only the GNN classifier choice. It is the end-to-end connection between Volatility artefacts, heterogeneous graphs, manifest governance, and explainable outputs in one reproducible system.

Chapter 3

Research Methodology

3.1 Research Design

This project follows an applied experimental design. It uses an existing implementation and its artefacts as the basis for dissertation writing. The method is not only theoretical. It includes a working pipeline that extracts, transforms, trains, evaluates, and reports on Windows memory-forensics data.

The research philosophy is pragmatic. The goal is to build a reproducible system that connects forensic extraction with graph learning. The evaluation therefore uses real project artefacts (CSVs, graphs, manifests, and analysis JSON files) rather than synthetic-only benchmarks.

The research process has five stages:

1. Collect or prepare Windows memory dump folders.
2. Extract memory artefacts using Volatility 3 plugins.
3. Build heterogeneous behaviour graphs from extracted CSV files.
4. Train and analyse graph neural network models.
5. Interpret results with evidence, uncertainty, and limitations.

Each stage produces files with defined contracts. This reduces silent errors and supports repeatability [10].

3.2 Data Sources

The main dataset is stored under the project `extracted_data` folder. The dataset manifest contains 30 rows. These rows represent malware-labelled and benign-labelled graph samples. The manifest includes fields such as sample ID, folder, label, family, node

count, edge count, maximum score, attack steps, injection counts, C2 connection counts, verdict, graph attributes, label signals, uncertainty status, and pipeline health flags.

The families and sample names include examples such as Cerber, Dharma, GandCrab, GoldenEye, WannaCry, PowerLoader, RedTail, Win32.BlackWorm, W32.MyDoom.A, and others. These names show the dataset has malware-family variety, but the dissertation does not treat one malware type as the full topic.

The corpus uses paired folders: **Family-WithVirus** and **Family-NoVirus**. This pairing supports controlled comparison within the same malware family context. It also highlights label difficulty when benign baselines still contain suspicious indicators.

The manifest also shows a key issue. Some NoVirus rows are marked as uncertain because they contain high or critical suspicious signals. In total, 14 of 15 benign-labelled rows are flagged uncertain in the current manifest. This does not mean the pipeline failed. It means benign baselines can contain suspicious-looking memory artefacts. The project handles this by recording uncertainty instead of hiding it.

3.3 Volatility 3 Extraction

Volatility 3 is used to extract structured artefacts from Windows memory dumps [6], [21]. The project server and scripts support these plugin outputs:

- windows.pslist
- windows.psscan
- windows.pstree
- windows.malfind
- windows.netscan
- windows.cmdline
- windows.dlllist
- windows.handles
- windows.threads
- windows.vadinfo
- windows.filescan
- windows.driverscan
- windows.ssdt

- `windows.registry.hivelist`

Table 3.1 maps plugins to forensic intent. This table helps explain why the graph schema includes process, memory, network, and kernel-related nodes.

Table 3.1: Volatility 3 plugins and primary forensic purpose in this project.

Plugin	Primary purpose
<code>pslist</code> , <code>psscan</code> , <code>pstree</code>	Process visibility and parent-child structure
<code>malfind</code> , <code>vadinfo</code>	Suspicious memory regions and VAD properties
<code>netscan</code>	Active network connections and endpoints
<code>cmdline</code> , <code>dlllist</code>	Execution context and module loading
<code>handles</code> , <code>threads</code>	Process interaction and execution units
<code>filescan</code> , <code>driverscan</code> , <code>ssdt</code>	File, driver, and syscall-table artefacts
<code>registry.hivelist</code>	Registry hive presence in memory

Each plugin gives a different view of the memory image. For example, `pslist` gives active process information, `psscan` can reveal process traces, `malfind` supports injected memory analysis, and `netscan` gives network connection information. The pipeline writes these outputs as CSV files for each sample folder.

Extraction is run per sample directory. The design keeps plugin outputs close to the graph build step so that provenance is clear: every edge and node can be traced back to a plugin CSV row or a derived rule outcome. This traceability is important when an analyst challenges a model score and wants to see the underlying forensic evidence chain.

3.4 Graph Construction Method

The graph construction stage converts plugin CSV files into a NetworkX graph [63]. The graph is later loaded by PyTorch Geometric [38]. Table 3.2 lists the main node types.

Table 3.2: Main node types in the heterogeneous memory graph.

Node type	Meaning
<code>process</code>	Running or recorded process instance
<code>thread</code>	Thread belonging to a process
<code>dll</code>	Loaded module
<code>memory_region</code>	Virtual memory region (VAD-style unit)
<code>network_conn</code>	Network connection object
<code>ip_address</code>	Remote or local IP endpoint
<code>handle</code>	Handle opened by a process
<code>driver</code>	Loaded driver
<code>kernel</code>	Kernel anchor node for system-level links

Table 3.3 summarises edge categories. Structural edges come from direct artefact relationships. Intent edges add security meaning for model input and analyst explanation.

Table 3.3: Representative edge types in the memory behaviour graph.

Edge type	Meaning
spawned_by, belongs_to	Process tree and ownership
loaded_into, allocated_in	DLL and memory placement
injected_into	Suspicious memory injection link
connects_from, connects_to	Network communication path
intent_c2, intent_injection	Behaviour intent for triage and learning
lolbin_execution_chain	Suspicious living-off-the-land style chain

This schema is important because malware behaviour is not represented only by one score. It is represented by relationships across a graph.

3.5 Feature Engineering

The dataset loader creates node features, edge features, and graph-level attributes. Node features include node type, suspicious flags, RWX memory indicators, MZ header indicators, external connection flags, private memory values, commit charge values, load counts, process context, and per-node edge-role counts.

Edge features are represented as categorical edge-type identifiers that can be embedded during GINE-style message passing [37]. Graph-level attributes include counts and derived signals such as graph size, graph density, suspicious behaviour flags, injection signals, C2 indicators, temporal chain counts, persistence counts, credential-access counts, privilege-escalation indicators, service mismatches, DLL trust anomalies, and graph motifs. These features are concatenated with graph embeddings during model inference.

The triage layer writes `graph_attr.json` with both a short legacy vector and an expanded attribute map. This design keeps backward compatibility while allowing richer graph-level features over time.

3.6 Model Methodology

The project supports several GNN backbones:

- **GIN**: useful for graph classification and structural differences [37].
- **GraphSAGE**: useful for neighbourhood aggregation and inductive learning [35].

- **GAT:** useful when attention over neighbours may improve representation [36].
- **GINE:** useful because edge types can be included through edge embeddings [37].

The binary model uses a GINE-style graph classifier for malware-versus-benign separation. The one-class models use GINE-style encoders to learn malware and benign conformity spaces. The analysis layer can then compare malware-pattern score, benign-conformity score, binary probability, heuristic risk, and uncertainty.

Graph-level readout uses multiple pooling operations (mean, max, and sum). The pooled representation is combined with graph attributes before the final classification or scoring head. This allows the model to use both learned structure and explicit forensic signals.

3.7 Evaluation Method

The evaluation is based on available project artefacts. The manifest records graph sizes, labels, verdicts, suspicious counts, and uncertainty fields. The analysis JSON files record graph summaries and evidence. The dissertation discusses these outputs carefully and avoids claiming production-level performance.

Important evaluation concerns are:

- small corpus size (30 graphs),
- possible family leakage,
- noisy benign baselines,
- uncertain labels,
- memory acquisition timing,
- plugin output variation,
- calibration quality,
- explainability needs.

Where classification metrics are reported, the project uses standard definitions. Precision is true positives divided by predicted positives. Recall is true positives divided by actual positives. F1 is the harmonic mean of precision and recall [54]. Calibration quality is discussed using reliability concepts from Guo et al. [9].

Grouped cross-validation is important because samples from the same family can be similar. If related samples appear in both training and test folds, the result can be too optimistic [45]. Family-grouped splits are therefore the recommended evaluation setting for any future model benchmarking on this corpus.

3.8 Ethics

Malware research must be handled safely. The project does not include malware binaries in the written dissertation. Memory dumps and extracted artefacts may contain sensitive system information. Therefore, raw memory data should be stored securely, shared only when allowed, and anonymised where possible. The research should follow university ethics guidance and legal rules, including guidance from standards such as ISO/IEC 27037 [27] and NIST incident response guidelines [11].

The written dissertation focuses on methods, aggregated statistics, and anonymised examples. It avoids publishing credentials, personal data, or live command lines that could identify individuals or organisations.

3.9 Data Collection and Sample Selection

Each sample in the corpus corresponds to a memory-forensics workspace folder under `extracted_data`. The folder naming pattern encodes both family and infection condition. A `WithVirus` folder is treated as malware-labelled. A `NoVirus` folder is treated as benign-labelled for experiments, while still allowing uncertain flags when triage signals are strong.

This pairing supports within-family comparison. For example, Cerber-`WithVirus` and Cerber-`NoVirus` share family context but may differ in process, memory, and network graphs. Such pairs are useful for error analysis because they show whether graph features capture infection-specific changes or only host-specific noise.

Sample selection was driven by availability of complete pipeline outputs. Rows with failed graph or analysis stages would be excluded from training. In the current manifest, all 30 rows report successful filter, graph, and analysis stages.

3.10 Labelling Policy and Uncertainty Handling

The project uses folder naming as the primary label source. This is simple and reproducible, but it is not perfect. Rule-based triage can assign `HIGH` or `CRITICAL` verdicts to `NoVirus` samples when suspicious artefacts appear in baseline memory.

The manifest therefore includes `uncertain` and `uncertain_reason` fields. Typical reasons include `benign_with_high_or_critical_verdict` and `benign_with_high_process_score`. This policy follows the idea that label quality should be reported rather than hidden [10].

For model training, uncertain benign rows can be handled in several ways: keep them as benign with sample weights, exclude them from training, or treat them as a third ambiguity class at inference time. The repository supports analyst-review states in the two-model analysis path so that these rows are not forced into a false sense of certainty.

3.11 Detailed Feature Groups

Node features. Node features combine categorical node type encodings with numeric forensic indicators. Suspicious flags summarise rule outcomes at node level. Memory-related nodes can include RWX and MZ-header indicators. Process-related nodes can include handle counts and lineage context.

Edge features. Edge types are encoded as integer identifiers for embedding lookup. This allows the model to treat `injected_into` differently from `belongs_to`. Edge features are especially important in GINE-style layers because they enter message functions directly.

Graph features. Graph features summarise global behaviour: size, density, suspicious counts, attack-step counts, injection and C2 indicators, temporal-chain counts, and motif counts. Many graph features come from `graph_attr.json` and therefore connect learning to transparent triage rules.

3.12 Training and Validation Protocol

The binary trainer uses a train-validation split with class-weight adjustment and early stopping. Positive-class weight can be tuned manually or through a small sweep when validation recall is below a target. This is useful when false negatives are more costly than false positives.

Grouped validation by `family` is the recommended setting for this corpus. A random split could place both Cerber-WithVirus and Cerber-NoVirus in different folds in unlucky ways, but family grouping reduces leakage more directly. Future work should report both random and grouped metrics to show how optimistic standard splits can be [45].

3.13 Threats to Validity

Internal validity. Pipeline bugs, schema drift, or incorrect edge construction could create artificial patterns. The project mitigates this with contracts, health flags, and manual inspection of sample graphs.

External validity. The corpus is small and may not represent all Windows versions, enterprise baselines, or malware families. Conclusions generalise cautiously.

Construct validity. Folder labels may not perfectly match ground truth infection state. Uncertainty flags make this explicit.

Conclusion validity. With only 30 graphs, strong claims about SOTA detection performance are not justified. The dissertation focuses on pipeline feasibility and evidence quality instead.

3.14 Research Timeline and Artefact Milestones

The applied methodology followed an iterative order:

1. stabilise Volatility extraction outputs for each sample folder;
2. implement graph build and verify node/edge counts;
3. add triage filtering and graph attributes;
4. generate analysis reports and attack-chain summaries;
5. build the dataset manifest for all successful samples;
6. implement dataset loader and GNN training scripts;
7. add binary and two-model analysis outputs with evidence fields;
8. integrate optional API scanning for interactive demos.

This order reduced rework because manifest rows were only created after graph and analysis health checks passed. It also made dissertation writing easier: each chapter maps to a stable artefact layer rather than to temporary experimental files.

3.15 Relation to docs.md and Code Contracts

The repository includes `docs.md` as a narrative project document and `docs/pipeline_contracts.md` as a contract reference. The dissertation chapters translate these materials into academic structure. When code and dissertation text differ, the manifest and contract files should be treated as authoritative for experiments because they reflect the latest pipeline behaviour.

Chapter 4

System Design

4.1 Overview

The system is designed as a pipeline from memory dump to model evidence. It is not only a model script. It includes extraction, graph construction, dataset creation, model training, analysis, and optional dashboard integration.

Figure 4.1 shows the end-to-end flow. Each stage writes artefacts that can be inspected independently. This separation is important for forensic research because analysts may trust extraction and triage outputs even when they question a single model score.

4.2 Extraction Layer

The extraction layer runs Volatility 3 plugins and saves outputs in each sample folder [6]. This design makes the pipeline repeatable. It also separates raw forensic extraction from machine learning. This is useful because the same CSV artefacts can be used for manual review, rule-based scoring, and model training.

The extraction layer is intentionally conservative. It prefers stable plugin outputs over experimental plugins that may change between Volatility versions. When a plugin file is missing for a sample, downstream stages record a pipeline health flag instead of failing silently.

4.3 Graph Layer

The graph layer builds a heterogeneous system graph. The graph is stored as `graph.pkl` for Python processing and `graph.json` for inspection or dashboard use. The graph structure is important because it keeps relationships between entities. For example, a memory region can be connected to the process it belongs to, and a network connection can be connected to the process that owns it.

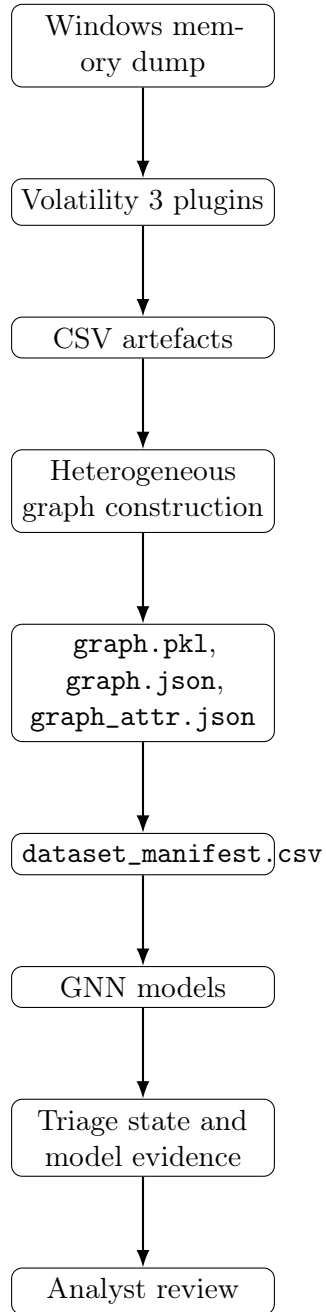


Figure 4.1: End-to-end memory-forensics and graph-learning pipeline.

The graph also includes behaviour-intent edges. These edges do not only show basic ownership. They add security meaning, such as injection intent, C2 intent, credential-access intent, persistence behaviour, or suspicious parent-child relations, which correspond to well-documented adversary techniques [3], [4], [30]. This improves the model input because it gives both structure and security context.

4.4 Triage and Graph Attributes

The rule-based triage layer creates `filtered_malicious.json` and `graph_attr.json`. These files contain structured signals such as suspicious PIDs, maximum process score, attack steps, injection counts, C2-style connections, and label signals. These attributes are not used to replace deep learning. They are used to support feature engineering and analyst explanation.

This hybrid design is practical. Rules are transparent and easy to inspect. GNN models can learn structural patterns that rules may miss. The final analysis can use both.

4.5 Dataset Manifest Design

The dataset manifest is the central dataset file. It links each graph folder to its label, family, graph counts, verdict fields, graph attributes, uncertainty status, and pipeline health fields. This supports reproducibility because a model run can be traced back to the exact folders and artefacts used [10].

The manifest also records whether graph creation, filtering, and analysis were successful. This prevents silent failures from being treated as valid training data. Table 4.1 summarises key artefact contracts used across the pipeline.

Table 4.1: Core artefact files and their role in the pipeline.

File	Producer	Purpose
<code>graph.pkl</code>	<code>build_graph.py</code>	Canonical graph object for ML and analysis
<code>filtered_malicious.json</code>	<code>filter_malicious.py</code>	Use evidence and suspicious PID list
<code>graph_attr.json</code>	<code>build_graph.py</code>	Graph-level attributes and label signals
<code>analysis_report.json</code>	<code>analyze_graph.py</code>	Per-sample graph summary and attack chain
<code>dataset_manifest.csv</code>	<code>build_dataset.py</code>	Dataset index for training and evaluation

4.6 Model Stack

The model stack has three main paths:

- **Binary GNN baseline:** predicts malware probability and supports calibrated thresholding.
- **Malware one-class model:** scores how close a sample is to learned malware behaviour.
- **Benign one-class model:** scores how close a sample is to learned benign behaviour.

Figure 4.2 shows how the two-model path combines conformity scores with heuristic risk to produce triage states.

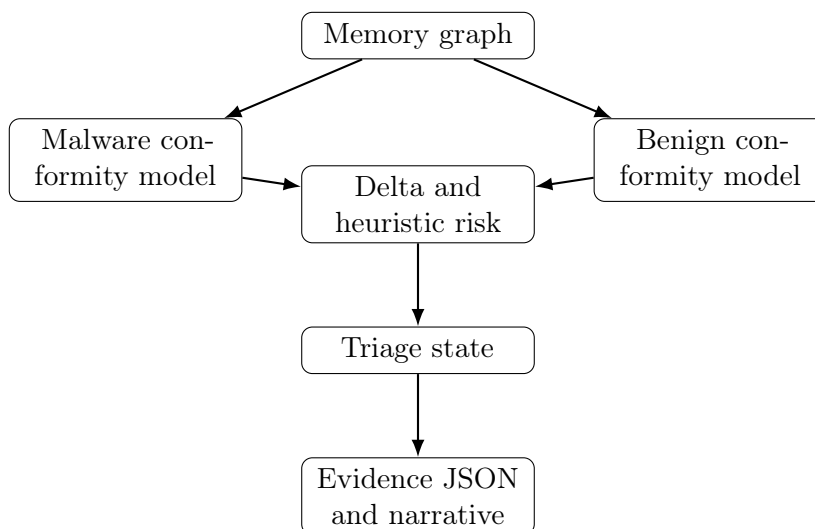


Figure 4.2: Two-model analysis flow from graph to structured evidence.

The two-model analysis combines malware conformity and benign conformity, drawing on one-class deep learning principles [51], [52]. This is useful because a memory snapshot can be suspicious but not clearly known malware. The output states include likely malicious, likely benign, anomalous unknown, and needs analyst review.

4.7 Evidence Design

The project is designed to preserve evidence. Analysis outputs include top graph attributes, top nodes, edge pairs, reasoning types, confidence fields, and a short narrative. This makes the result more useful for incident response, consistent with the argument that interpretability must be treated as a core requirement [55]. Explanation methods such as GNNExplainer [41] provide a direction for future work. An analyst can start from the model output and then check the related graph artefacts or raw CSV rows.

4.8 Interface Between Triage Rules and Learning

The triage layer and the GNN layer are connected but not identical. Triage rules operate on interpretable thresholds and pattern checks. GNNs operate on learned combinations of structure and attributes. The system design allows three interaction modes:

1. **Rules only:** analysts rely on verdicts and JSON evidence without model scores.
2. **Rules + binary model:** analysts use probability and calibrated states with evidence.
3. **Rules + two-model scores:** analysts compare malware and benign conformity to detect ambiguity.

This flexibility supports different organisational maturity levels. A lab may use rules only while collecting more data. A research team may enable GNN paths when checkpoints and manifests are stable.

4.9 Dashboard and Operational Integration

The repository also contains server and dashboard components. The API supports memory dump upload, plugin execution, graph pipeline execution, and model scanning [64]. The dashboard client can request a model scan for a dump session when `graph.pkl` exists. This shows a possible operational route for the research: memory dump upload, Volatility extraction, graph build, model scan, and analyst review in a user interface.

The operational path is optional for dissertation evaluation. The core research evidence comes from the batch pipeline and manifest, not from live UI demos.

4.10 Design Rationale by Layer

4.10.1 Why separate extraction and learning

Extraction and learning are separated so that forensic outputs remain valid even when model code changes. If a GNN architecture is replaced, the underlying CSV and graph artefacts should still be usable for manual review. This separation also helps debugging: when a score is unexpected, analysts can inspect triage JSON and plugin outputs without retraining models.

4.10.2 Why use both rules and GNNs

Rules encode transparent heuristics that security teams already understand. GNNs can combine many weak signals across structure. The hybrid design avoids two extremes:

pure rules that miss complex patterns, and pure deep learning that behaves like a black box. Graph attributes from rules are included as features, so the model can learn when to trust or discount heuristic signals.

4.10.3 Why graph-level classification

Node-level classification would require node labels that are expensive to produce in memory forensics. Graph-level labels (malware vs benign for the snapshot) match the folder-level supervision available in this corpus. Graph-level outputs also match operational questions such as “should this memory image be prioritised for deeper review?”

4.11 Failure Handling and Health Flags

The manifest records `filter_ok`, `graph_ok`, and `analyze_ok`. If any step fails, the row should not enter training silently. This design follows good MLOps practice for research code [65].

Common failure modes include missing plugin CSV files, empty process lists, graph build exceptions, and analysis timeouts on very large graphs. Health flags make these cases visible in evaluation tables and help explain unexpected metric changes between manifest versions.

4.12 Security and Privacy Considerations in Design

Memory dumps may contain passwords, personal documents, browser history, and other sensitive data. The system design therefore assumes secure storage, restricted access, and controlled sharing. The dashboard/API path should run only in trusted lab environments.

For dissertation reporting, aggregated statistics and anonymised examples are preferred over raw command lines or host-identifying strings. This reduces ethical risk while still allowing technical discussion of malware behaviour.

Chapter 5

Implementation

5.1 Development Environment

The project is implemented mainly in Python. It uses Volatility 3 for forensic extraction [6], pandas for CSV processing [61], NetworkX for graph construction [63], PyTorch and PyTorch Geometric for deep learning [58], [38], and FastAPI for the backend service [64]. The dashboard side uses TypeScript and web API routes to interact with the model scan flow.

The implementation follows a script-oriented research codebase. Each major step has a command-line entry point. This makes it easier to rerun one stage after fixing a bug without rebuilding the entire corpus.

5.2 Main Repository Components

The main implementation components are:

- `auto_vol.py`: runs Volatility extraction over memory dumps.
- `build_graph.py`: builds heterogeneous graphs from CSV artefacts.
- `filter_malicious.py`: produces triage signals and suspicious evidence.
- `analyze_graph.py`: creates graph-level analysis reports.
- `build_dataset.py`: builds the dataset manifest.
- `dataset.py`: loads graphs into PyTorch Geometric format.
- `model.py`: defines GIN, GraphSAGE, GAT, and GINE classifiers.
- `one_class_gnn.py`: defines one-class GINE utilities.

- `train_binary_model.py`: trains the binary GNN baseline.
- `train_malware_model.py` and `train_benign_model.py`: train one-class models.
- `analyze_binary_model.py`: produces calibrated binary model analysis.
- `analyze_two_model.py`: combines malware and benign model outputs.
- `model_scan.py`: runs model scanning for one upload-session graph.
- `server.py`: exposes the extraction, graph, IOC, and scan workflow through an API.

5.2.1 Extraction and graph build

`auto_vol.py` orchestrates plugin execution and writes `windows_*.csv` files into each sample directory. `build_graph.py` reads these tables, normalises identifiers, and creates nodes and edges. The script also writes `graph.pkl` and `graph.json`. The JSON form supports manual inspection and dashboard rendering.

`filter_malicious.py` runs after an initial graph exists. It identifies suspicious PIDs and writes triage metadata used later as graph attributes. This step connects rule-based forensic reasoning with machine-learning features.

5.2.2 Dataset and model code

`build_dataset.py` walks the corpus and writes `dataset_manifest.csv`. The manifest is the single index used by training and evaluation scripts. `dataset.py` converts each graph into a PyTorch Geometric `Data` object with consistent feature shapes.

`model.py` defines the GNN backbones and graph-level classifier head. The implementation uses global pooling and concatenates graph attributes before the final layer. `one_class_gnn.py` provides encoders and scoring utilities for the malware and benign conformity models.

5.3 Data Contracts

The project uses clear file contracts. Each sample can produce:

- `windows_*.csv` plugin outputs,
- `graph.pkl`,
- `graph.json`,
- `filtered_malicious.json`,

- `graph_attr.json`,
- `analysis_report.json`.

At dataset level, the main output is `dataset_manifest.csv`. This file is used by the dataset loader and training scripts. Keeping these files consistent is important because machine learning results depend on stable input shape and meaning.

If a contract changes, the project expects coordinated updates in the loader, manifest builder, and analysis scripts. Hidden schema drift is a common source of technical debt in ML systems [65].

5.4 Node and Edge Implementation

The graph schema is based on real memory-forensics entities. Processes, threads, DLLs, memory regions, network connections, handles, drivers, and kernel objects become nodes. Relationships become typed edges. The graph therefore represents a system snapshot as a connected behaviour structure.

This approach is stronger than a simple count table because it preserves relationships. For example, the model can learn that injected memory connected to a process with unusual network activity may be more important than an isolated count of memory regions.

Edge typing also allows the GINE encoder to treat different relations differently. This is useful when injection edges and ownership edges carry different security meaning.

5.5 Why GINE and What Alternatives Were Considered

5.5.1 Why GINE over untyped convolutions

Forensic graphs are heterogeneous: edge types encode distinct semantics (parent-child, DLL load, injection, network ownership, behaviour-intent links). Standard GCN or mean-aggregation layers treat all edges similarly and can blur type-specific signals. GINE extends GIN with edge-conditioned message functions so each relation type contributes separately [37]. That matches the design goal: preserve forensic meaning in the message-passing path rather than collapsing relations into unlabeled adjacency.

5.5.2 Why graph-level classification, not node labels

Memory dumps in this corpus provide snapshot-level supervision (`WithVirus` vs `NoVirus`). Node-level malware labels are not available without expensive manual annotation. Graph-

level classification answers the operational question “should this memory image be prioritised?” and aligns with manifest labels, at the cost of ignoring which specific process caused the label.

5.5.3 Supported backbones and expected trade-offs

`model.py` implements GIN, GraphSAGE, GAT, and GINE with the same pooling head and graph-attribute concatenation. Table 6.2 (generated by `scripts/generate_eval_latex.py` after grouped CV) is the intended comparison point. Until CV JSON files exist, cells remain placeholders.

- **GIN:** strong baseline for graph classification; no edge-type conditioning.
- **GraphSAGE:** inductive neighbourhood sampling; useful if graphs grow beyond current corpus scale.
- **GAT:** attention over neighbours; higher compute on large graphs (many memory-region nodes).
- **GINE:** default for binary and one-class paths because edge types are informative.

5.5.4 What did not work well in practice

Three trade-offs appeared during development. **Small n :** with 30 graphs, validation folds are unstable; family grouping is necessary but reduces effective training size per fold. **Train time vs graph size:** samples above 20,000 nodes dominate batch memory; training epochs are costly relative to tabular baselines on manifest features alone. **Heuristic features vs pure learning:** graph attributes already encode strong triage signals; GNN gains may be modest unless structure adds information beyond `graph_attr.json`—an open question for the tabular-vs-GNN comparison in Chapter 6.

5.6 GNN Implementation

The GNN models perform message passing over the graph [8]. Each graph is converted into PyTorch Geometric data [38] with:

- `x` for node features,
- `edge_index` for graph connectivity,
- `edge_attr` for edge-type IDs,
- `graph_attr` for graph-level features,

- `batch` for mini-batch processing.

The model then creates a graph-level representation using pooling operations. The implementation uses mean, max, and sum pooling. The pooled graph vector is joined with graph-level attributes before final classification or scoring.

5.6.1 Training configuration

The binary trainer supports class-weight tuning, early stopping, and optional learning-rate scheduling. Default training uses multiple epochs with validation-based model selection. The script can sweep positive-class weights to balance recall and specificity on small validation folds.

One-class trainers learn embedding-space boundaries for malware-only or benign-only subsets. These models are later used by `analyze_two_model.py` to compare conformity scores on new graphs.

5.7 Evidence and Inference Implementation

The analysis scripts do more than return a score. They enrich nodes and edges with evidence metadata, align features with checkpoint expectations, apply calibration where needed, compute uncertainty fields, and write structured JSON so analysts can audit outputs.

The two-model analysis produces fields such as:

- malware pattern score,
- benign conformity score,
- delta score,
- triage state,
- reasoning types,
- behavioural findings,
- model evidence,
- narrative explanation.

`model_scan.py` exposes a single-graph inference path for the API workflow. This is useful for upload-session analysis in the dashboard prototype.

5.8 Implementation Challenges

Three challenges appeared repeatedly during development.

Missing or partial plugin output. Not every sample had identical CSV coverage. The graph builder had to tolerate absent files while still recording health flags in the manifest.

Graph-size variation. The corpus includes graphs from 86 nodes to more than 27,000 nodes. Batch training therefore required careful memory use and padding strategies.

Noisy benign labels. Many NoVirus folders produced HIGH or CRITICAL rule-based verdicts. The implementation response was to add **uncertain** flags and analyst-review states instead of forcing clean benign labels.

5.9 Reproducibility

Reproducibility is supported by the manifest, saved model metadata, stable graph attributes, and fixed file contracts [10]. A future reader can inspect which samples were used, which graph attributes were present, and which steps completed successfully. This is important in cybersecurity research because small implementation changes can alter results, a problem known as hidden technical debt in machine learning systems [65].

Appendix A lists recommended commands to rebuild the dataset and regenerate dissertation figures.

5.10 Per-Module Implementation Notes

5.10.1 `build_graph.py`

The graph builder reads plugin CSV files and maps rows to node dictionaries. Processes are linked with parent-child relations when **pstree** or equivalent information is available. DLL and memory-region nodes attach to process nodes with typed edges. Network connections link to owning processes and remote IP nodes.

The builder also computes derived indicators such as suspicious process scores and attack-step counts used later in triage. When enrichment is enabled, behaviour-intent edges are added from rule outcomes rather than from raw plugin rows alone.

5.10.2 `filter_malicious.py`

The filter stage identifies suspicious PIDs and writes `filtered_malicious.json`. This file includes metadata used by both graph attribute generation and analyst review. The filter is intentionally readable: an analyst can open the JSON file and see which PIDs triggered suspicion without reading model code.

5.10.3 analyze_graph.py

The analysis stage produces `analysis_report.json` with graph summaries, attack-chain steps, and structured suspicious-node listings. Version 3 of the analyser enriches outputs used by the dissertation case studies. The WannaCry example in Chapter 6 comes directly from this file format.

5.10.4 dataset.py and training scripts

The dataset loader reads `dataset_manifest.csv`, loads each `graph.pkl`, and converts graphs to PyTorch Geometric batches. Feature alignment checks help prevent silent mismatches between training and inference checkpoints.

`train_binary_model.py` trains the supervised baseline. `train_malware_model.py` and `train_benign_model.py` train one-class encoders. `analyze_binary_model.py` and `analyze_two_model.py` generate JSON outputs for single samples or batches.

5.10.5 server.py and model_scan.py

The server exposes endpoints for extraction, graph build, and scanning workflows. `model_scan.py` supports session-based inference when a graph already exists for an uploaded dump. This path is useful for interactive demos but is not required for batch evaluation on the 30-sample corpus.

5.11 Hyperparameters and Training Defaults

Table 5.1 summarises main training defaults used in the binary trainer. These values are implementation defaults rather than fully tuned optimal settings. They are reported here for reproducibility.

Table 5.1: Representative binary training defaults (`train_binary_model.py`).

Setting	Typical value / behaviour
Epochs	Configurable (default in script)
Optimiser	Adam-style update with weight decay
Learning rate schedule	Cosine annealing supported
Early stopping	Enabled on validation F1 stall
Class weight	Auto or manual positive-class weight
Validation split	Hold-out split from manifest rows
Output	Checkpoint and metadata JSON

One-class trainers follow a similar pattern but optimise embedding-space objectives rather than binary cross-entropy. Analysis scripts load checkpoints and align feature

dimensions before inference. When checkpoint files are absent, analysis scripts cannot produce model scores and the evaluation must rely on triage artefacts alone.

5.12 Performance and Resource Considerations

Large graphs dominate memory use. Samples with more than 20,000 nodes require efficient batching and may limit hyperparameter search. The project mitigates this by graph-level classification (one label per graph) rather than node-level classification across the full corpus.

CPU preprocessing time is also significant. Volatility extraction and graph construction can take much longer than one training epoch on small data. For research planning, artefact generation should be treated as a first-class timeline item, not as a one-time preliminary step.

5.13 Testing and Validation During Development

During development, samples were inspected manually at three checkpoints: after CSV extraction, after graph build, and after analysis JSON generation. This manual review caught issues such as missing plugin files, unexpected empty graphs, and extreme node counts. Manual review is not scalable to thousands of hosts, but it was valuable for a 30-sample research corpus.

Regression checks should include: (1) manifest row count stable, (2) no missing health flags, (3) graph node counts within expected order of magnitude for family, and (4) analysis verdicts present. The dissertation figures provide a quick visual regression check for corpus-level shifts when graph rules change.

Chapter 6

Evaluation and Discussion

6.1 Central Empirical Finding

The strongest result from the current corpus is not a detection F1 score. It is the systematic mismatch between benign folder labels and rule-based triage signals.

In `dataset_manifest.csv` ($n=30$), **14 of 15** benign-labelled (NoVirus) rows are marked `uncertain=True`. Reasons include HIGH or CRITICAL verdicts, elevated process scores, injection counts, and C2-style connection counts despite benign labels. Only the W32.MyDoom.A.-NoVirus row is not uncertain. Malware-labelled rows are not flagged uncertain in the manifest.

Figure 6.1 summarises the split. Figure 6.2 places it in the wider label balance. This pattern means folder naming is an imperfect proxy for ground truth: “clean” baselines in memory forensics can still contain suspicious artefacts from security tools, prior activity, or capture timing.

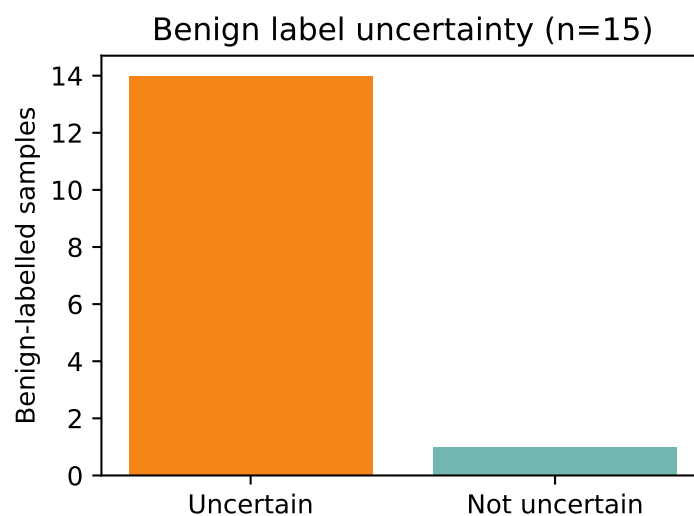


Figure 6.1: Benign-labelled samples: uncertain vs not uncertain ($n=15$).

For machine learning, uncertain benign rows should be excluded, down-weighted, or reported separately in evaluation. Treating all NoVirus folders as definitively benign will inflate false conclusions about separability.

6.2 Proof-of-Concept Scope

This chapter evaluates whether the pipeline produces consistent artefacts, meaningful graphs, and auditable evidence on a small research corpus. It does **not** claim production malware detection performance. Quantitative ML tables use placeholders until grouped cross-validation is run and checkpoints are archived (see §6.5).

6.3 Descriptive Corpus Statistics

Table 6.1 summarises manifest-level scale statistics (from `figures/dataset_stats.json`). Graph size spans more than two orders of magnitude; both labels include very large graphs.

Table 6.1: Corpus summary statistics (30 manifest rows).

Statistic	Min	Median	Max	Notes
Nodes (all)	86	18,036	27,362	MyDoom pair smallest
Edges (all)	7	21,544	32,556	
Malware-labelled	15	—	—	No uncertain flags
Benign-labelled	15	—	—	

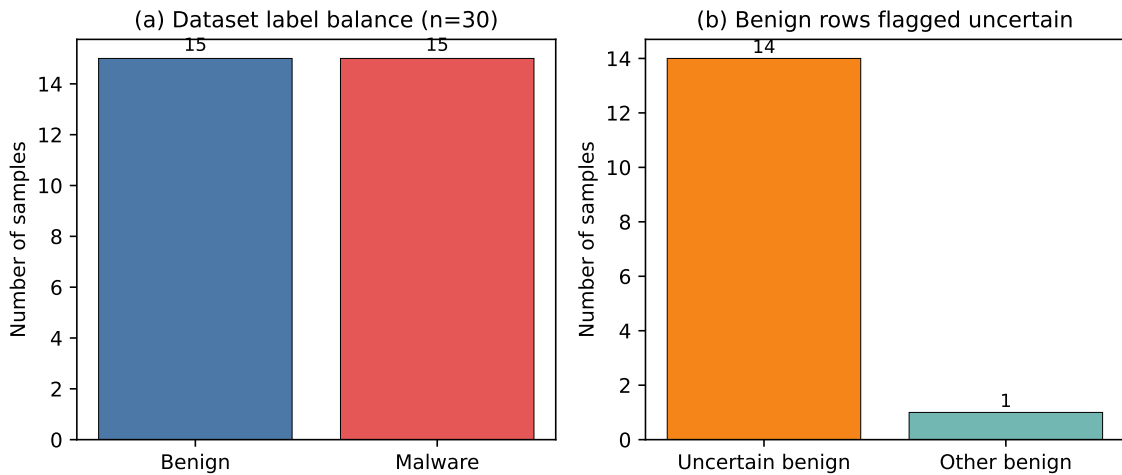


Figure 6.2: Dataset composition: malware vs benign labels and uncertain benign subset.

All listed rows completed filter, graph, and analysis stages (`filter_ok`, `graph_ok`, `analyze_ok`), giving a stable artefact base for discussion.

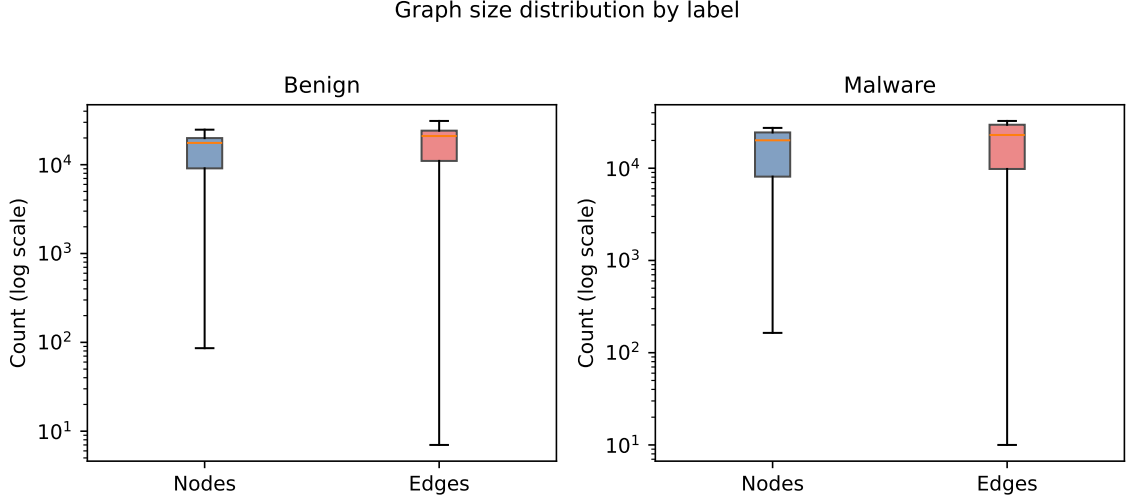


Figure 6.3: Node and edge counts by label (log scale).

6.4 Rule-Based Triage Analysis

Rule-based signals in the manifest show false-positive pressure on NoVirus rows. Figure 6.4 compares mean injection count, C2 connections, and maximum process score by label. Malware-labelled means are not uniformly higher than benign-labelled means for every signal.

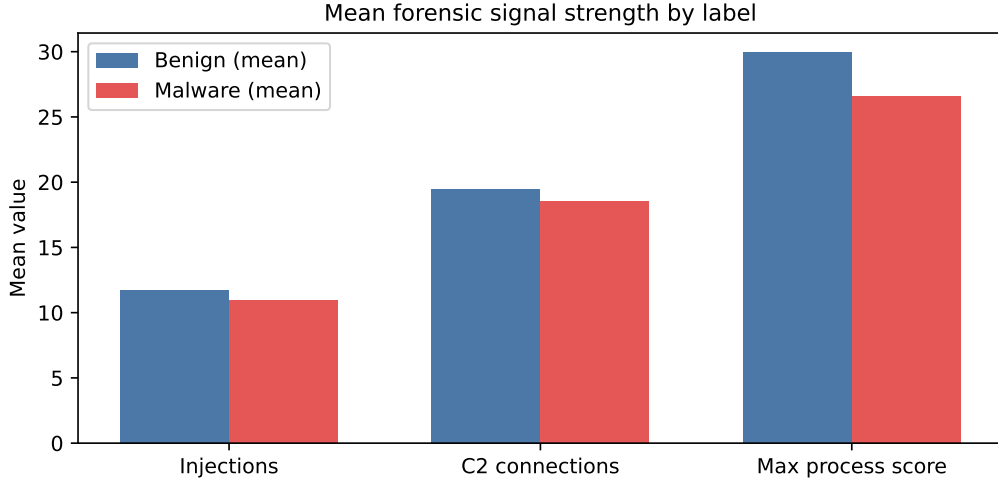


Figure 6.4: Mean forensic signal values by label (manifest-derived).

Figure 6.5 shows that most samples receive HIGH or CRITICAL verdicts. Only the MyDoom pair reports LOW verdicts. Benign-labelled rows therefore often receive the same verdict bands as malware-labelled rows, which explains the uncertain flags.

Figure 6.6 shows each malware family contributes paired WithVirus and NoVirus rows. Family-grouped splits are mandatory for honest evaluation [45].

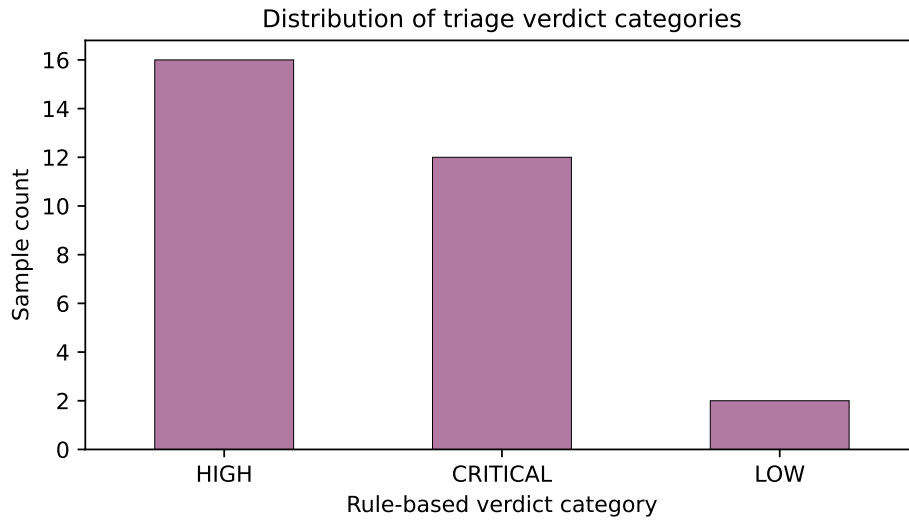


Figure 6.5: Rule-based triage verdict categories across the corpus.

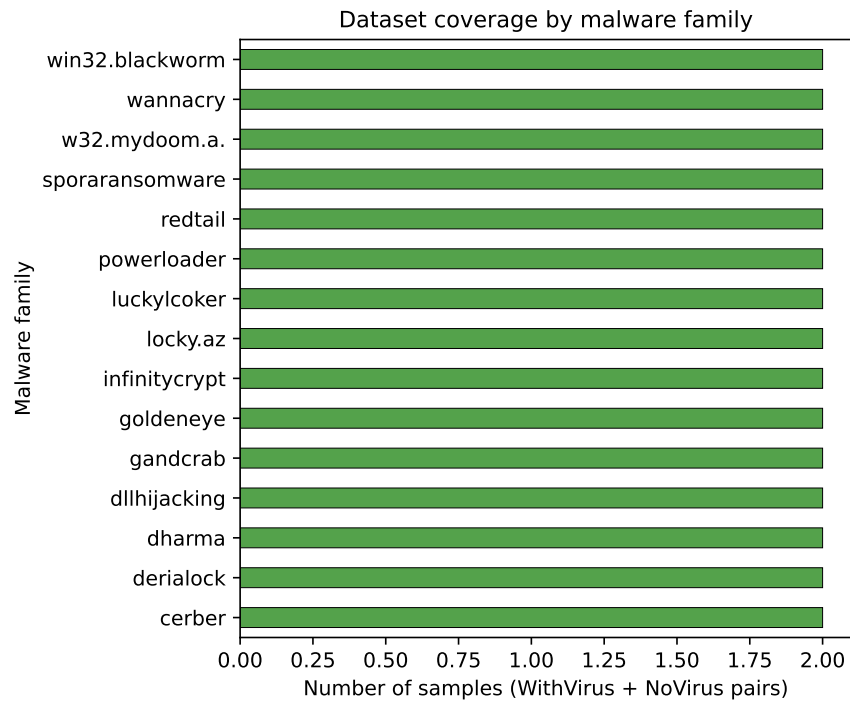


Figure 6.6: Manifest rows per malware family.

6.5 Quantitative Machine Learning Evaluation

Grouped evaluation is implemented in `train.py` (StratifiedGroupKFold by family), `train_binary_model.py`, and `scripts/tabular_baselines.py`. Saved `results_*.json` files were not present in the repository at writing time; Table 6.2–6.4 therefore show “–” placeholders. After running experiments, regenerate tables with:

```
python train.py extracted_data/dataset_manifest.csv --cv stratified_group \
  --group-by family --model gine --epochs 80 --seed 42
python scripts/tabular_baselines.py
python scripts/generate_eval_latex.py
```

6.5.1 GNN backbone comparison (StratifiedGroupKFold by family)

Table 6.2: Held-out GNN results (populate after running `train.py`).

Model	Precision	Recall	F1	ROC-AUC
GIN	–	–	–	–
SAGE	–	–	–	–
GAT	–	–	–	–
GINE	–	–	–	–

6.5.2 Tabular baselines (manifest graph-level features)

Table 6.3: Tabular baseline CV (`tabular_baselines.py`).

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	–	–	–	–
Random Forest	–	–	–	–

6.5.3 Cross-validation protocol

6.5.4 Evaluation figures (placeholders until predictions exist)

When `outputs/predictions.csv` is produced from grouped CV, `scripts/generate_eval_latex.py` can render confusion-matrix and ROC PDFs (`fig_confusion_matrix.pdf`, `fig_roc_curve.pdf`). Until then, only manifest-derived figures (including Figure 6.1) are included.

A natural comparison is tabular classifiers on `graph_attr` features versus full-graph GNNs on the same manifest version. If tabular models match GNNs, relational structure

Table 6.4: Evaluation protocol used for reported metrics.

Setting	Value
Split strategy	StratifiedGroupKFold
Group column	<code>family</code>
Folds	5 (capped by number of families)
Random seed	42
Exclude uncertain benign	Optional (<code>-exclude-uncertain</code>)

may add little on this small, heuristic-rich corpus; if GNNs improve under grouped CV, that supports the graph design.

6.6 Failure Cases and Edge Families

6.6.1 W32.MyDoom.A. pair (very small graphs)

W32.MyDoom.A.-NoVirus (86 nodes) and W32.MyDoom.A.-WithVirus (164 nodes) are outliers. Both report LOW verdicts and zero injections; neither benign row is uncertain. Models trained mainly on large graphs may underfit or overfit on tiny topologies—pooling and normalisation should be checked on this pair.

6.6.2 Cerber pair (similar scale, divergent C2 counts)

Cerber-WithVirus (7,904 nodes) and Cerber-NoVirus (9,382 nodes) have comparable scale. The malware row reports 24 C2 connections and `lolbin_c2_found=True`; the benign row reports 7 C2 connections and is uncertain. Both share HIGH verdicts and six injections in the manifest. A GNN may not separate these folders if features emphasise shared injection structure more than family-specific infection cues.

6.6.3 GoldenEye pair (high injection on both rows)

GoldenEye-WithVirus has the highest injection count (41). GoldenEye-NoVirus still reports 8 injections, 13 hidden processes, and a CRITICAL verdict. This is among the clearest uncertain-benign examples: triage treats both snapshots as highly suspicious.

6.6.4 WannaCry case study

WannaCry-WithVirus (8,098 nodes) shows injection-related attack-chain steps and ransom-note signals. WannaCry-NoVirus (7,590 nodes) reports nine injections, a HIGH verdict, and `uncertain=True`. Figure 6.7 shows memory regions dominate node types—typical for VAD-heavy graphs.

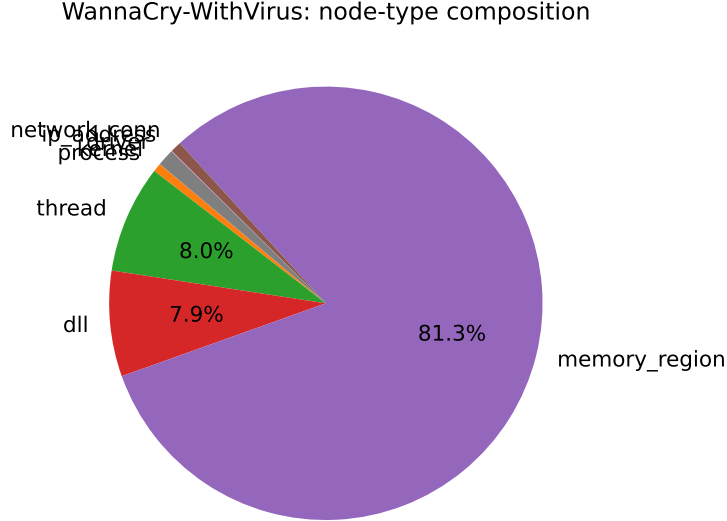


Figure 6.7: Node-type composition for WannaCry-WithVirus.

Table 6.5: WannaCry manifest comparison (selected fields).

Folder	Label	Nodes	Injections	Uncertain
WannaCry-WithVirus	Malware	8,098	7	No
WannaCry-NoVirus	Benign	7,590	9	Yes

6.6.5 When a GNN may not help

GNNs add value when relational structure carries signal beyond graph-level heuristics. They may add little when: (1) labels are dominated by triage noise; (2) graphs differ mainly in size, not infection topology; (3) train and test families are too few for stable grouped metrics; or (4) edge and node features already encode the decision rules. The uncertain-benign prevalence pushes toward analyst-review workflows and two-model conformity scores rather than hard binary automation.

6.7 Binary and Two-Model Paths

The binary GNN path supports calibrated probabilities and ambiguous states [9]. On noisy labels, a single threshold can encode triage noise rather than infection state. The two-model path compares malware and benign conformity [51], surfacing cases where both scores are elevated—consistent with uncertain benign rows.

Saved checkpoints were not archived in the repository at writing time. Implementation capability is documented; grouped metrics require the user to train, save *.pt files, and regenerate LaTeX tables.

6.8 Explainability

The project records top graph attributes, important nodes, and edge pairs in JSON evidence. This is not a complete explanation method [55], but it exceeds a single opaque score. GNNExplainer [41] remains future work.

6.9 What Results Prove and Do Not Prove

Proven on current artefacts:

- Volatility artefacts can be converted into typed graphs from 86 to 27,362 nodes.
- Rule-based triage produces HIGH/CRITICAL verdicts on most benign-labelled rows (14/15 uncertain).
- Manifest governance supports reproducible, family-aware evaluation design.
- Evidence JSON and uncertainty fields are implementable alongside GNN scores.

Not proven:

- Superior detection accuracy versus production AV or EDR on enterprise hosts.
- Generalisation to unseen malware families without grouped CV on larger n .
- That GNN structure beats strong tabular baselines on the same manifest (pending experiments).
- Robustness to adversarial graph manipulation [46].

6.10 Limitations

- Small corpus ($n=30$) with high metric variance under grouped CV.
- Noisy benign labels (14/15 uncertain).
- Family-level pairing limits independent test families.
- Capture timing and Windows build affect plugin output.
- Heuristic graph attributes couple rules and learning.
- Missing archived checkpoints and prediction CSVs at submission time.

Sommer and Paxson [45] and Sculley et al. [65] apply: evaluation design and pipeline debt can dominate headline metrics.

6.11 Practical Value

The pipeline supports evidence-led triage [22]: organise plugin output, surface rule verdicts, attach GNN scores as a secondary signal, and export JSON for analyst review. The uncertain-benign finding is operationally relevant—teams should expect HIGH verdicts on baseline images without treating them automatically as labelling errors.

6.12 Summary

1. Benign-labelled rows frequently exhibit HIGH/CRITICAL triage signals (14/15 uncertain)—the central empirical finding.
2. Graph construction scales from tiny to very large memory snapshots.
3. Rule-based triage alone cannot define benign status from folder names.
4. Grouped CV and tabular baselines are implemented; numeric ML tables await experiment runs.
5. Structured evidence improves usefulness over a single probability.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This dissertation implemented a proof-of-concept pipeline for graph-based malware triage from Windows memory forensics: Volatility 3 extraction, typed memory-snapshot graphs, graph-level features, GNN scoring (GIN, GraphSAGE, GAT, GINE), and structured evidence with uncertainty fields [6], [38].

The primary empirical finding is label-signal mismatch on benign baselines: **14 of 15** benign-labelled manifest rows are marked uncertain because rule-based triage still reports HIGH or CRITICAL activity. Folder names (`NoVirus`) are an imperfect proxy for ground truth. This result should be reported openly in any memory-forensics ML study, not treated as a labelling failure to hide.

Secondary contributions are reproducible engineering artefacts: manifests, data contracts, grouped evaluation scripts, tabular baselines, and JSON evidence for analyst review. Together they show that graph-based representation can organise memory-forensics evidence at scale, while detection metrics on $n=30$ remain feasibility evidence until larger corpora and archived checkpoints are available.

7.2 Answers to Research Questions

RQ1: How can memory artefacts be represented as a graph? Volatility CSV outputs map to typed nodes and edges (processes, memory regions, DLLs, network objects, intent relations). The manifest records scale and triage metadata per sample.

RQ2: Which entities and relationships are most useful? Process-memory and process-network links are central; intent edges align behaviour with ATT&CK-style patterns [3]. Memory-region nodes dominate graph size in case studies such as WannaCry.

RQ3: How can GNNs support detection with readable evidence? Binary and two-model paths combine learned scores with graph attributes, top nodes, and JSON

evidence. Uncertain benign flags motivate analyst-review states rather than forced binary decisions.

RQ4: What limitations appear on a small corpus? High graph-size variance, 14/15 uncertain benign rows, and family pairing require grouped splits and modest claims [45]. Placeholder metric tables document what remains to be run experimentally.

7.3 Reflection on the Research Aim

The aim—to design and evaluate a graph-based deep learning pipeline for memory-forensics malware detection—was met at the pipeline and evidence level. Extraction, graph construction, manifests, models, and analyst-facing outputs are implemented. Full quantitative detection evaluation is partial because checkpoints and `results_*.json` files were not archived; Chapter 6 states that boundary explicitly.

The work does not claim GNNs outperform all alternatives on all malware. It demonstrates feasibility, foregrounds benign-label uncertainty, and specifies the next experiments (grouped CV, tabular comparison, calibration plots).

7.4 Contributions

- Volatility 3 extraction workflow and artefact contracts;
- heterogeneous memory-snapshot graph schema;
- graph-level feature engineering linked to triage;
- GNN and tabular evaluation scripts with family-grouped CV;
- structured evidence and uncertainty reporting;
- dataset manifest for reproducibility;
- optional dashboard upload-session scanning.

7.5 Future Work

- Expand corpus with clean enterprise baselines and known-good capture times;
- Run and archive grouped CV for GIN/SAGE/GAT/GINE and tabular baselines; populate Table 6.2;
- Add GNNExplainer [41] or similar graph explanations;
- Test on families held out entirely from training;

- Improve label curation with analyst feedback on uncertain rows;
- Monitor feature drift across manifest versions;
- Compare against gradient boosting and operational EDR telemetry [60], [1].

7.6 Implications for Practice

Practitioners should treat HIGH or CRITICAL verdicts on benign-labelled baselines as expected when memory contains security tools or noisy host state. Workflow: acquire memory, run plugins, build graphs, inspect triage JSON, then use GNN scores as a secondary signal. Educators can use manifests and figures without requiring enterprise-scale labelled data.

7.7 Final Statement

Graph-based deep learning is a plausible direction for memory-forensics malware triage because system behaviour is relational. The strongest outcome of this project is a reproducible, explainable path from memory dump to evidence—with honest reporting that benign labels and triage signals often disagree. Detection metrics on the current corpus should support feasibility claims only until grouped benchmarks on larger data replace the placeholders in Chapter 6.

Chapter 8

Reflection

8.1 Project Learning

This project improved my understanding of both memory forensics and machine learning. At the start, the work looked like a malware detection task. During development, it became clear that data representation was the most important part. If the memory artefacts are not extracted and organised correctly, the model cannot learn useful patterns.

I also learned that dissertation scope must match the evidence. The project contains ransomware families, but the research topic is broader: graph-based malware detection from Windows memory forensics. Keeping this focus improved the clarity of the writing and the evaluation.

8.2 Technical Reflection

The most difficult technical challenge was joining many Volatility outputs into one stable graph. Each plugin gives different columns and different levels of detail. The graph builder had to handle missing files, different object types, and noisy evidence. This taught me that cybersecurity machine learning depends strongly on data engineering [65].

Another challenge was uncertainty. Some NoVirus samples had suspicious features and HIGH or CRITICAL verdicts. Instead of ignoring this, the project records uncertainty in the manifest. This is a more honest approach and fits real forensic work, where evidence is often mixed.

Graph-size variation was also difficult for training. A graph with fewer than 200 nodes and a graph with more than 25,000 nodes appear in the same dataset. This required careful batching and realistic expectations about model stability on small data.

8.3 Research Reflection

The project also improved my research writing. The correct dissertation topic is broader than ransomware. Rewriting the work around graph-based deep learning made the argument stronger and more accurate. It also helped separate the dataset examples from the main research contribution.

Working with IEEE-style references improved my literature discipline. Each major claim should be linked to prior work or to project artefacts. This is similar to forensic reporting, where each conclusion should be linked to evidence.

8.4 Professional Reflection

This project is relevant to digital forensics, incident response, malware analysis, and security engineering. It helped me understand how a research prototype can be designed for practical analyst use. The most valuable lesson is that detection tools should explain their evidence and limitations clearly [55], [22].

If this system were deployed in a SOC environment, I would present it as a triage assistant. Analysts would still validate important cases manually. The tool would help them prioritise which memory dumps deserve deeper review.

8.5 Closing Reflection

The project shows that good cybersecurity research needs both technical implementation and careful communication. A model is not enough by itself. The data pipeline, graph design, evidence output, uncertainty handling, and documentation all matter. This learning will be useful for future work in malware analysis and defensive security.

I would advise future students on similar projects to lock artefact contracts early, publish a manifest for every experiment, and report uncertain labels openly. These habits improve trust in the final dissertation and in the underlying code.

8.6 Skills Developed

Through this project I developed practical skills in Volatility-based artefact interpretation, graph schema design, PyTorch Geometric modelling, and technical writing for dual audiences (academic examiners and security practitioners). I also learned to use manifests and JSON evidence files as first-class research outputs, not as secondary logs.

8.7 What I Would Do Differently

If starting again, I would collect more clean benign baselines from known-good enterprise images and document acquisition timing for every dump. I would also archive trained checkpoints with manifest version IDs from the first successful training run. That would make Chapter 6 stronger because model metrics could be reported alongside corpus statistics.

Finally, I would add automated grouped cross-validation scripts earlier in the project. This would reduce the risk of optimistic results when related family folders leak between train and test splits.

8.8 Personal Outcome

Completing this dissertation improved my confidence in building end-to-end security research systems. I now understand that writing, implementation, and evaluation must be planned together. A strong model without manifests, contracts, and evidence outputs would not meet the needs of forensic practitioners. That lesson will guide my approach in future cyber security roles and research projects.

Appendix A

Project Artefacts and Reproducibility

A.1 Project Artefact Map

Artefact	Purpose
windows_*.csv	Volatility 3 plugin output for each sample
graph.pkl	NetworkX graph used by graph processing and model loading
graph.json	JSON graph representation for inspection and UI use
filtered_malicious.json	Rule-based triage evidence and graph attributes
graph_attr.json	Graph-level attributes and label signals
analysis_report.json	Per-sample graph analysis report
dataset_manifest.csv	Dataset index used for training and evaluation
*.pt model files	Saved PyTorch model checkpoints
*_meta.json	Model metadata and calibration information

A.2 Manifest Column Glossary

Column	Meaning
sample_id	Numeric identifier for each graph row
folder	Directory name for the sample
label	0 = NoVirus (benign), 1 = WithVirus (malware)
family	Malware family tag used for grouped evaluation
nodes, edges	Graph size statistics
max_score	Highest suspicious process score from triage
injections, c2_conns	Counts of injection and C2-style indicators

Column	Meaning
verdict	Rule-based triage verdict string
uncertain	Flag for label or baseline uncertainty
filter_ok, graph_ok, analyze_ok	Pipeline health indicators

A.3 Core Plugin Set

The project uses the following main Volatility 3 plugins: `pslist`, `psscan`, `pstree`, `malfind`, `netscan`, `cmdline`, `dlllist`, `handles`, `threads`, `vadinfo`, `filescan`, `driverscan`, `ssdt`, and `registry.hivelist`.

A.4 Reproducibility Commands

The following commands rebuild dataset artefacts, run evaluation, and regenerate dissertation tables from the repository root:

```
python build_dataset.py
python scripts/generate_dissertation_figures.py

# Grouped CV -- GNN backbones (family-held-out)
python train.py extracted_data/dataset_manifest.csv --cv stratified_group \
  --group-by family --model gine --epochs 80 --seed 42

# Binary GINE baseline
python train_binary_model.py extracted_data/dataset_manifest.csv \
  --output-model models/binary_gnn.pt --output-meta models/binary_meta.json

# Tabular baselines (Logistic Regression, Random Forest)
python scripts/tabular_baselines.py

# Regenerate LaTeX tables and eval figures
python scripts/generate_eval_latex.py

python analyze_binary_model.py
python analyze_two_model.py
```

To compile the dissertation PDF:

```
cd latex_dissertation
latexmk -pdf main.tex
```

A.5 Acronyms

API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
C2	Command and Control
CFG	Control Flow Graph
CSV	Comma-Separated Values
DLL	Dynamic Link Library
GAT	Graph Attention Network
GIN	Graph Isomorphism Network
GINE	Graph Isomorphism Network with Edge features
GNN	Graph Neural Network
IOC	Indicator of Compromise
ML	Machine Learning
NDSS	Network and Distributed System Security
PyG	PyTorch Geometric
RWX	Read-Write-Execute (memory protection)
SOC	Security Operations Centre
VAD	Virtual Address Descriptor

A.6 Manifest Excerpt (First Five Rows)

ID	Label	Folder	Nodes	Uncertain
01	0	Cerber-NoVirus	9382	True
02	1	Cerber-WithVirus	7904	False
03	0	DLLHijacking-NoVirus	18477	True
04	1	DLLHijacking-WithVirus	27362	False
05	0	DeriaLock-NoVirus	24787	True

A.7 Figure Generation

Dissertation figures are produced from project data using:

```
python scripts/generate_dissertation_figures.py
```

Outputs are written to `latex_dissertation/figures/` as PDF files included in Chapter 6.

Appendix B

Dataset Statistics Summary

This appendix summarises corpus-level statistics derived from `dataset_manifest.csv` at dissertation writing time.

B.1 Label and uncertainty counts

- Total samples: 30
- Malware-labelled (WithVirus): 15
- Benign-labelled (NoVirus): 15
- Benign rows with `uncertain=True`: 14
- Benign rows with `uncertain=False`: 1 (W32.MyDoom.A.-NoVirus)

B.2 Graph size ranges

- Nodes: minimum 86, maximum 27,362, median approximately 18,036
- Edges: minimum 7, maximum 32,556, median approximately 21,544

B.3 Families represented

The manifest includes paired samples for families including Cerber, DLLHijacking, DeriaLock, Dharma, GandCrab, GoldenEye, InfinityCrypt, Locky.AZ, LuckyLcoker, PowerLoader, Red-Tail, SporaRansomware, W32.MyDoom.A., WannaCry, and Win32.BlackWorm. Each family contributes one WithVirus and one NoVirus row in the current manifest.

B.4 Interpretation notes

These statistics are descriptive, not performance metrics. They show that the corpus is balanced at the label level but unbalanced in difficulty and graph size. Any future model evaluation should report manifest version, family-grouped splits, and uncertainty-aware error analysis.

B.5 Recommended Reporting Checklist for Future Experiments

When extending this project, the following items should appear in any results table or dissertation chapter:

1. manifest row count and label balance;
2. number and percentage of uncertain benign rows;
3. min/median/max graph sizes;
4. family-grouped split description;
5. whether checkpoints and calibration files are archived;
6. whether evaluation uses rules only, tabular ML, or full GNN;
7. example evidence JSON excerpts for true positives and false positives.

This checklist helps keep future results comparable with the baseline established here.

Appendix C

Glossary of Key Terms

Memory dump A captured image of volatile memory from a host at a point in time.

Volatility plugin A module that extracts a specific artefact type from a memory dump.

Heterogeneous graph A graph with multiple node and edge types.

Graph attribute A feature vector summarising the whole graph, not a single node.

Triage A first-pass risk ranking to prioritise analyst review.

WithVirus / NoVirus Folder naming convention indicating malware-infected vs benign comparison sample in this corpus.

Uncertain label Manifest flag when benign-labelled data still shows strong suspicious indicators.

Conformity score One-class similarity score measuring closeness to a learned class embedding space.

Evidence field Structured output explaining which attributes, nodes, or edges influenced a score.

Appendix D

Volatility Plugin Notes (Extended)

The following notes summarise how plugins support the graph schema. `pslist` and `psscan` help identify process nodes and hidden process indicators. `pstree` supports parent-child process edges. `malfind` and `vadinfo` support memory-region nodes and injection-related edges. `netscan` supports network connection and IP nodes. `cmdline` and `dlllist` enrich process context. `handles` and `threads` add interaction structure. `filescan`, `driverscan`, and `ssdt` support driver and kernel-related nodes. Registry hive listing supports registry-related context where present.

These plugins do not need to succeed equally on every sample. The pipeline records health flags when stages fail so that incomplete samples are visible in the manifest rather than silently polluting training data.

Readers comparing this dissertation to `docs.md` should note that the LaTeX version is the submission-oriented document with IEEE references and figures, while `docs.md` remains a working project narrative in the repository root.

Bibliography

- [1] D. Ucci, L. Aniello, and R. Baldoni, “Survey of machine learning techniques for malware analysis,” *Computers & Security*, vol. 81, pp. 123–147, 2019.
- [2] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, “A survey on automated dynamic malware-analysis techniques and tools,” *ACM Computing Surveys*, vol. 44, no. 2, pp. 1–42, 2012.
- [3] MITRE, “Mitre attack enterprise matrix,” <https://attack.mitre.org/>, 2026.
- [4] —, “Process injection, technique t1055,” <https://attack.mitre.org/techniques/T1055/>, 2026.
- [5] M. H. Ligh, A. Case, J. Levy, and A. Walters, *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*. Indianapolis, IN, USA: Wiley, 2014.
- [6] The Volatility Foundation, “Volatility 3 documentation,” <https://volatility3.readthedocs.io/>, 2026.
- [7] M. Russinovich, D. A. Solomon, A. Ionescu, and P. Yosifovich, *Windows Internals, Part 1*, 7th ed. Redmond, WA, USA: Microsoft Press, 2017.
- [8] W. L. Hamilton, *Graph Representation Learning*. San Rafael, CA, USA: Morgan & Claypool, 2020.
- [9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1321–1330.
- [10] S. N. Goodman, D. Fanelli, and J. P. A. Ioannidis, “What does research reproducibility mean?” *Science Translational Medicine*, vol. 8, no. 341, 2016.
- [11] National Institute of Standards and Technology, “Computer security incident handling guide,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-61 Revision 2, 2012.
- [12] Cybersecurity and Infrastructure Security Agency, “Stopransomware guide,” <https://www.cisa.gov/stopransomware>, 2023.

- [13] —, “Malware analysis report resources,” <https://www.cisa.gov/news-events/cybersecurity-advisories>, 2026.
- [14] I. Santos, Y. K. Peña, J. Devesa, and P. G. Bringas, “N-grams-based file signatures for malware detection,” in *Proceedings of the 11th International Conference on Enterprise Information Systems*, 2009, pp. 317–320.
- [15] M. Christodorescu, S. Jha, S. A. Seshia, D. Song, and R. E. Bryant, “Semantics-aware malware detection,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2005, pp. 32–46.
- [16] A. Moser, C. Kruegel, and E. Kirda, “Limits of static analysis for malware detection,” in *Proceedings of the 23rd Annual Computer Security Applications Conference*, 2007, pp. 421–430.
- [17] U. Bayer, P. M. Comporetti, C. Hlauschek, C. Kruegel, and E. Kirda, “Scalable, behavior-based malware clustering,” in *Proceedings of the Network and Distributed System Security Symposium*, 2009.
- [18] C. Willems, T. Holz, and F. Freiling, “Toward automated dynamic malware analysis using cwsandbox,” *IEEE Security & Privacy*, vol. 5, no. 2, pp. 32–39, 2007.
- [19] H. Yin, D. Song, M. Egele, C. Kruegel, and E. Kirda, “Panorama: Capturing system-wide information flow for malware detection and analysis,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007, pp. 116–127.
- [20] K. Kent, S. Chevalier, T. Grance, and H. Dang, “Guide to integrating forensic techniques into incident response,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-86, 2006.
- [21] The Volatility Foundation, “Volatility 3,” <https://github.com/volatilityfoundation/volatility3>, 2026.
- [22] A. Case and G. G. Richard, “Memory forensics: The path forward,” *Digital Investigation*, vol. 20, pp. 23–33, 2017.
- [23] S. L. Garfinkel, “Digital forensics research: The next 10 years,” *Digital Investigation*, vol. 7, pp. S64–S73, 2010.
- [24] M. Russinovich, A. Ionescu, D. A. Solomon, and P. Yosifovich, *Windows Internals, Part 2*, 7th ed. Redmond, WA, USA: Microsoft Press, 2021.
- [25] Microsoft, “Windows internals documentation,” <https://learn.microsoft.com/en-us/sysinternals/resources/windows-internals>, 2026.
- [26] M. Graziano, A. Lanzi, and D. Balzarotti, “Hypervisor memory forensics,” in *Proceedings of the 17th International Symposium on Research in Attacks, Intrusions and Defenses*, 2013, pp. 21–40.

- [27] ISO/IEC, “Iso/iec 27037:2012: Guidelines for identification, collection, acquisition and preservation of digital evidence,” International Organization for Standardization, Geneva, Switzerland, 2012.
- [28] E. Casey, *Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet*, 3rd ed. Waltham, MA, USA: Academic Press, 2011.
- [29] B. Carrier, *File System Forensic Analysis*. Boston, MA, USA: Addison-Wesley, 2005.
- [30] MITRE, “Data encrypted for impact, technique t1486,” <https://attack.mitre.org/techniques/T1486/>, 2026.
- [31] B. Perozzi, R. Al-Rfou, and S. Skiena, “Deepwalk: Online learning of social representations,” in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [32] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [33] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” 2013, arXiv:1301.3781.
- [34] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proceedings of the International Conference on Learning Representations*, 2017, arXiv:1609.02907.
- [35] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 1024–1034.
- [36] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in *Proceedings of the International Conference on Learning Representations*, 2018, arXiv:1710.10903.
- [37] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proceedings of the International Conference on Learning Representations*, 2019, arXiv:1810.00826.
- [38] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019, arXiv:1903.02428.
- [39] T. K. Rusch, M. M. Bronstein, and S. Mishra, “A survey on oversmoothing in graph neural networks,” in *Proceedings of the International Conference on Learning Representations*, 2023, arXiv:2303.10993.
- [40] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, “Measuring and relieving the over-smoothing problem for graph neural networks from the topological view,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 3438–3445, 2020.

- [41] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “Gnnexplainer: Generating explanations for graph neural networks,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [42] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [43] W. Jin, J. Ma, Y. Wang, G. Tang, J. Zhang, and J. Tang, “Adversarial attacks on graph neural networks via meta learning,” *Proceedings of the International Conference on Learning Representations*, 2022, arXiv:2102.08456.
- [44] H. Dai, H. Li, T. Tian, X. Huang, L. Wang, J. Zhu, and L. Song, “Adversarial attack on graph structured data,” *Proceedings of the International Conference on Machine Learning*, pp. 1115–1124, 2018, foundational graph attack; cited for threat model.
- [45] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [46] G. Apruzzese, B. Biggio, M. Conti *et al.*, “Sok: Real attackers don’t compute gradients,” in *IEEE Symposium on Security and Privacy*, 2023, pp. 3149–3167.
- [47] F. Pierazzi, M. Andriushchenko, B. Biggio *et al.*, “Intriguing properties of adversarial ml attacks in the problem space,” in *IEEE Symposium on Security and Privacy*, 2020, pp. 1332–1349.
- [48] X. Zhan *et al.*, “A survey of graph neural networks for malware detection,” *Computers & Security*, vol. 116, p. 102663, 2022.
- [49] K. Rieck, T. Holz, C. Willems, P. Dussel, and P. Laskov, “Learning and classification of malware behavior,” in *Proceedings of the 5th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, 2008, pp. 108–125.
- [50] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, K. Rieck, and C. Siemens, “Drebin: Effective and explainable detection of android malware in your pocket,” in *Proceedings of the Network and Distributed System Security Symposium*, 2014.
- [51] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Muller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 4393–4402.
- [52] G. Pang, C. Shen, L. Cao, and A. van den Hengel, “Deep learning for anomaly detection: A review,” *ACM Computing Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [53] S. Axelsson, “The base-rate fallacy and its implications for the difficulty of intrusion detection,” in *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 1999, pp. 1–7.

- [54] J. Davis and M. Goadrich, “The relationship between precision-recall and roc curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240.
- [55] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” 2017, arXiv:1702.08608.
- [56] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should i trust you? explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [57] A. Shrikumar, P. Greenside, and A. Kundaje, “Learning important features through propagating activation differences,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 3145–3153.
- [58] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [59] PyTorch Geometric Team, “Pytorch geometric documentation,” <https://pytorch-geometric.readthedocs.io/>, 2026.
- [60] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [61] W. McKinney, “Data structures for statistical computing in python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.
- [62] C. R. Harris *et al.*, “Array programming with numpy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [63] A. Hagberg, P. Swart, and D. S. Chult, “Exploring network structure, dynamics, and function using networkx,” in *Proceedings of the 7th Python in Science Conference*, 2008, pp. 11–15.
- [64] FastAPI, “Fastapi documentation,” <https://fastapi.tiangolo.com/>, 2026.
- [65] D. Sculley *et al.*, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [66] PyTorch Foundation, “Pytorch documentation,” <https://pytorch.org/docs/stable/index.html>, 2026.
- [67] National Institute of Standards and Technology, “Security and privacy controls for information systems and organizations,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-53 Revision 5, 2020.
- [68] H. Wu *et al.*, “Graph neural networks in cybersecurity: A survey,” *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–35, 2023.